



深圳河套學院

Shenzhen Loop Area Institute

Natural Language Processing and Large Language Models

Xiaojun Quan and Zhengjun Yue

Shenzhen Loop Area Institute

Autumn, 2026

Lecture 3

Language Models

Learning Objectives

Explain sequence probability as repeated next-token prediction.

1. Compare feedforward neural and recurrent language models.
2. Trace hidden-state updates and parameter sharing through a basic RNN.
3. Explain how RNNs are trained and why their limitations motivate Transformer.

Outline

- Language Modeling
- Recurrent Neural Networks
- RNN Language Modeling and Training
- RNN Applications and Seq2seq

RNN Limitations and the Bridge to Attention

Practice & Discussion

Takeaways

Outline

- ❑ **LANGUAGE MODELING**
 - ❑ Recurrent Neural Networks
 - ❑ RNN Language Modeling and Training
 - ❑ RNN Applications and Seq2seq
- RNN Limitations and the Bridge to Attention
- Practice & Discussion
- Takeaways

Probabilistic Language Models

□ Motivation: compute the probability of a sequence of words.

■ Machine Translation (机器翻译):

- $P(\text{strong winds tonight}) > P(\text{large winds tonight})$

■ Spelling Correction (拼写纠正)

- The office is about fifteen **minuets** from my house

$P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$

■ Speech Recognition (语音识别)

- $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

■ Summarization (自动摘要), question-answering (问答系统), etc.

Probabilistic Language Models

- **Goal:** compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

- **Related task:** probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- A model that computes either of these:

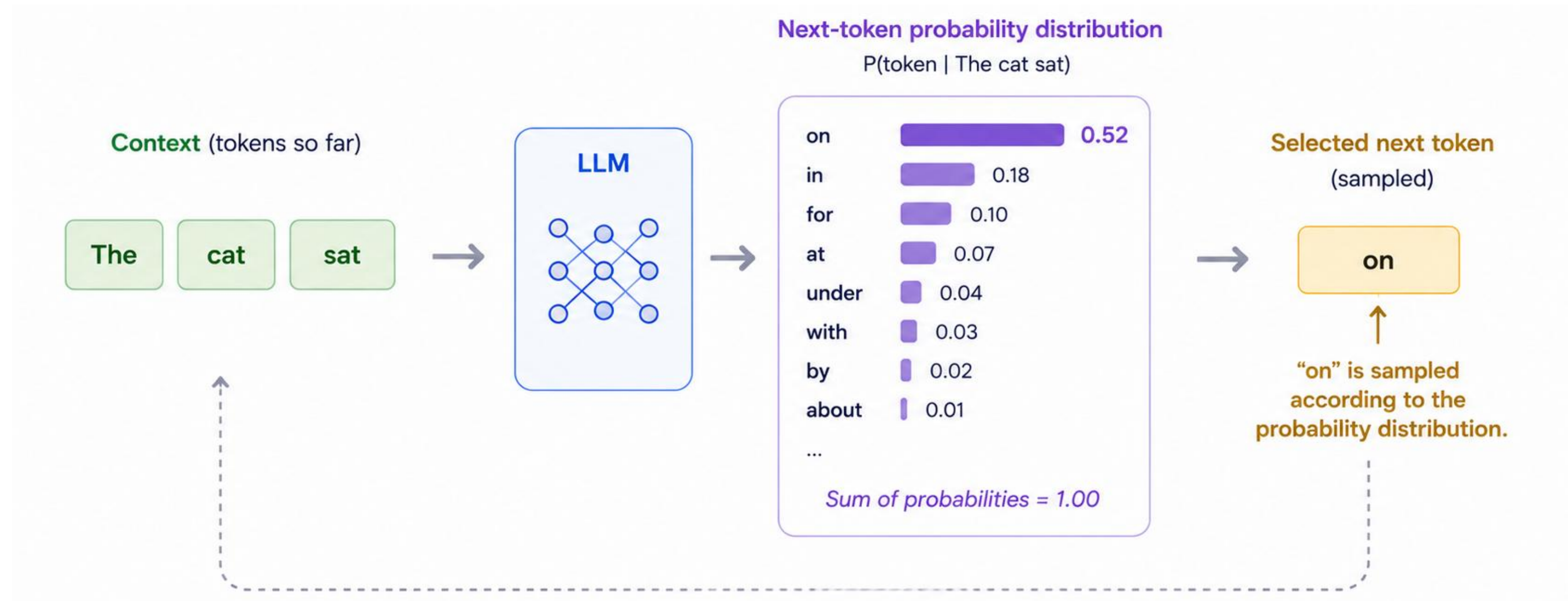
$P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$ is called a **language model (语言模型)**.

A Language Model Predicts **What Comes Next**

□ Given a context, the model assigns probabilities to possible next tokens.

■ Several continuations may be plausible at the same time.

The output is a probability distribution, not one stored answer.



How to compute $P(W)$?

□ How to compute this joint probability:

○ $P(\text{its, water, is, so, transparent, that})$

□ Intuition: Use the Chain Rule (链式法则) of probability.

The Chain Rule (链式法则)

- Recall the definition of conditional probabilities

$$P(A,B) = P(A)P(B|A)$$

- More variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

- The Chain Rule in general

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

Chain Rule for joint probability of words

$$P(w_1 w_2 \cdots w_n) = \prod_i P(w_i \mid w_1 w_2 \cdots w_{i-1})$$

$$\begin{aligned} P(\text{“its water is so transparent”}) = \\ P(\text{its}) \times P(\text{water} \mid \text{its}) \times P(\text{is} \mid \text{its water}) \\ \times P(\text{so} \mid \text{its water is}) \times P(\text{transparent} \mid \text{its water is so}) \end{aligned}$$

Note: In language modeling, we compute the joint probability from **front to back** to reflect the sequential nature of language.

Why Use Log Probabilities?

- ❑ Sequence probabilities multiply many numbers and cause **underflow**.
- Taking logs converts products into sums.

1 Sequence probability becomes a product

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

Example: $0.2 \times 0.1 \times 0.05 \times 0.01 = \mathbf{0.00001}$

Multiplying many probabilities quickly gives extremely small numbers.



2 Log turns products into sums

$$\log P(x_1, \dots, x_T) = \sum_{t=1}^T \log P(x_t | x_{<t})$$

Example: $\log(0.2 \times 0.1 \times 0.05 \times 0.01) =$
 $\log 0.2 + \log 0.1 + \log 0.05 + \log 0.01$



Sums are easier to compute, analyze, and optimize than long products.

3 Same ranking, easier optimization

$$\arg \max_y P(y | x) = \arg \max_y \log P(y | x)$$

- The log function is monotonic, so the best sequence does not change.
- Logs make gradients and training objectives more convenient.



4 Negative log-likelihood as the training loss

$$\mathcal{L} = - \sum_{t=1}^T \log P(x_t | x_{<t})$$

or equivalently

$$\mathcal{L} = - \log P(x_1, \dots, x_T)$$

- Higher probability $\rightarrow$ lower loss
- Avoids numerical underflow
- Makes token losses additive

More probable sentence

$$P = 0.01$$

$$\mathcal{L} = -\log(0.01) = \mathbf{4.61}$$

<

Less probable sentence

$$P = 0.000001$$

$$\mathcal{L} = -\log(0.000001) = \mathbf{13.82}$$

Higher probability leads to lower loss.

Probability Is Not the Same as Truth

High Probability $\neq$ Factual Correctness

Frequent continuation (but actually incorrect)

The capital of Australia is **Paris** .

$P(\text{next token} = \text{"Paris"}) = 0.18$

- ✓ **Likely:** Yes (common in training data)
- ✓ **Well-formed:** Yes
- ✗ **True:** **No**
(The capital of Australia is Canberra.)

Less common but correct continuation

The capital of Australia is **Canberra** .

$P(\text{next token} = \text{"Canberra"}) = 0.01$

- ✓ **Likely:** Less common
- ✓ **Well-formed:** Yes
- ✓ **True:** Yes

How likely it sounds, how correct the language is, and whether it is true are different things.

Likelihood

How likely the sentence is according to the model's learned patterns.

Language correctness

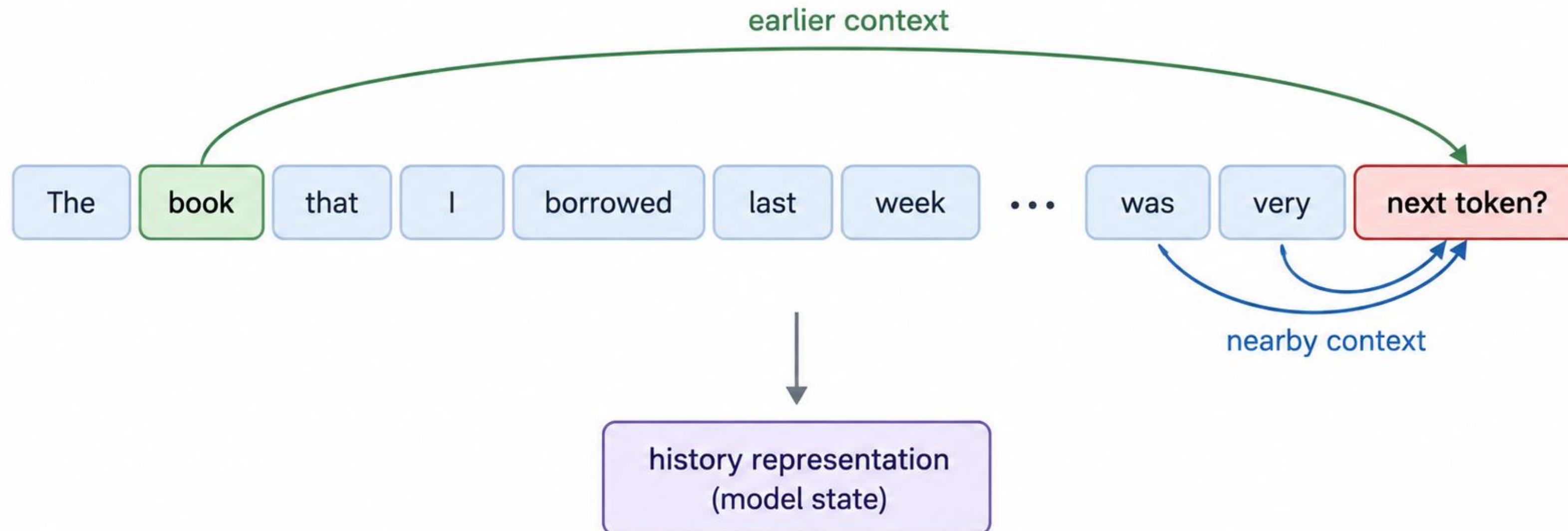
Whether the sentence follows the rules of the language.

Truth

Whether the sentence matches the real world.

The Challenge Is Using History

- ❑ **The next token may depend on nearby words or information much earlier in the sequence.**
 - A useful model must decide what previous information matters.
 - The main architectural question is how that history is represented.



Outline

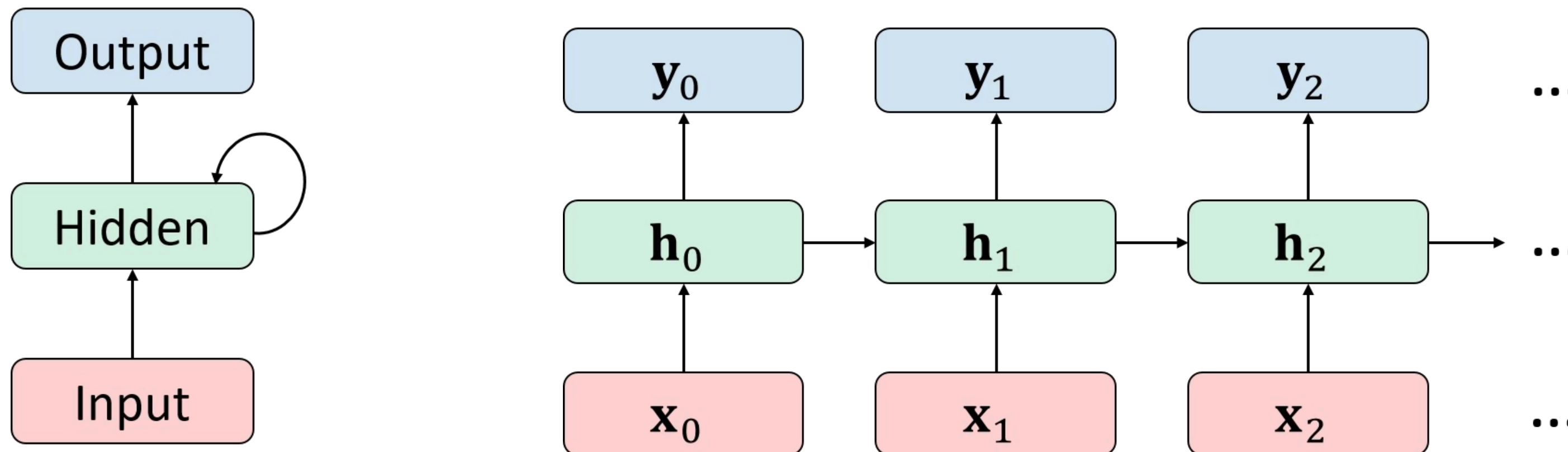
- Language Modeling
 - **RECURRENT NEURAL NETWORKS**
 - RNN Language Modeling and Training
 - RNN Applications and Seq2seq
- RNN Limitations and the Bridge to Attention
- Practice & Discussion
- Takeaways

Neural Language Models (LMs)

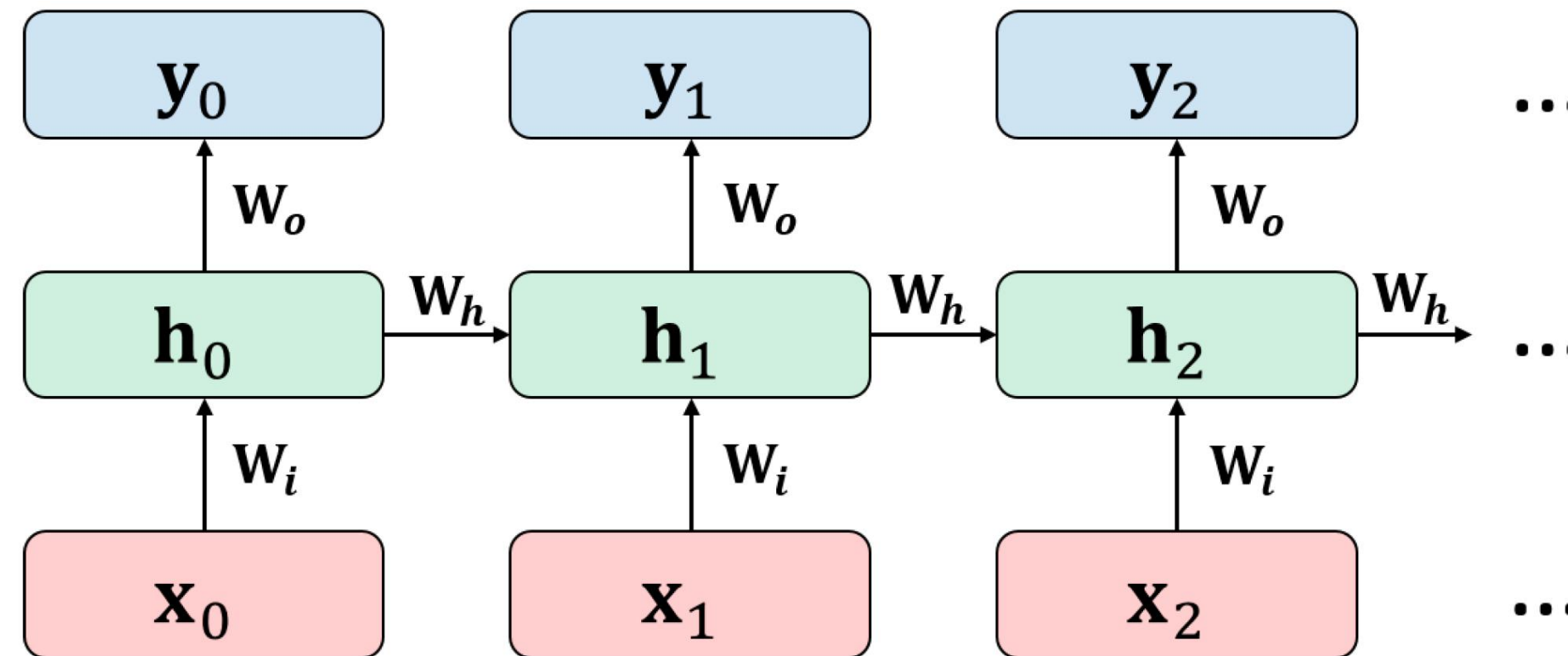
- ❑ **Language Modeling:** Essentially calculating the probability of the next word in a sequence given some history.
- ❑ State-of-the-art neural LMs are based on powerful neural networks like Recurrent Neural Networks and Transformers.

Recurrent Neural Networks: **Basics**

- ❑ **Recurrent Neural Network** (RNN) is a class of neural network architecture primarily used for processing sequential data.
- ❑ RNNs have a “memory” function, allowing them to make predictions **based on the historical information** of an input sequence.



Recurrent Neural Networks: Basics



$$\mathbf{h}_t = g(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t + \mathbf{b}_h)$$

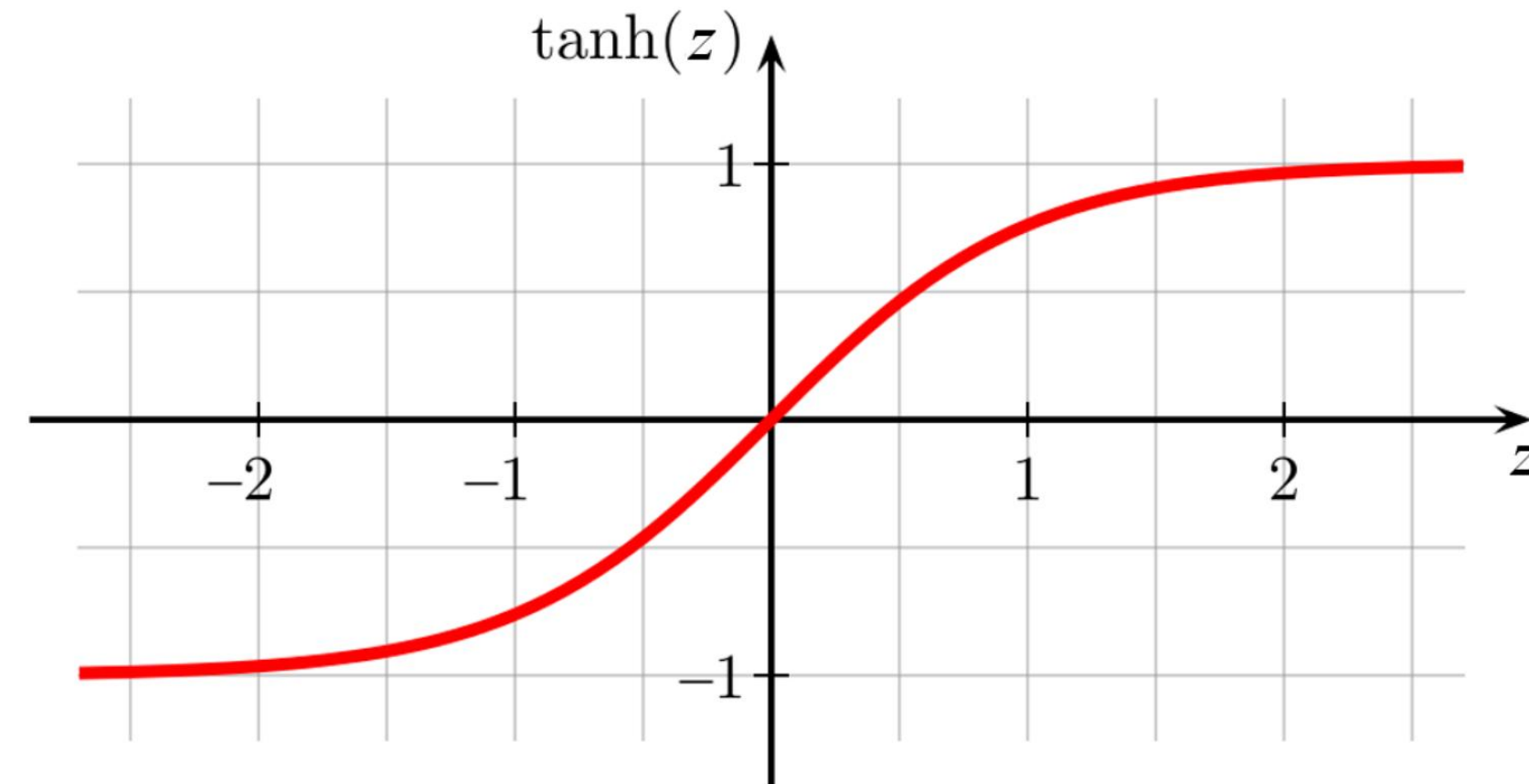
$$\mathbf{y}_t = \sigma(\mathbf{W}_o \mathbf{h}_t + \mathbf{b}_o)$$

- $g(\cdot)$ activation function, usually *tanh*; $\sigma(\cdot)$: *softmax* function.
- The parameters used at different time steps are *shared*.

Why use *tanh* in RNN?

Why RNNs use *tanh* activation function?

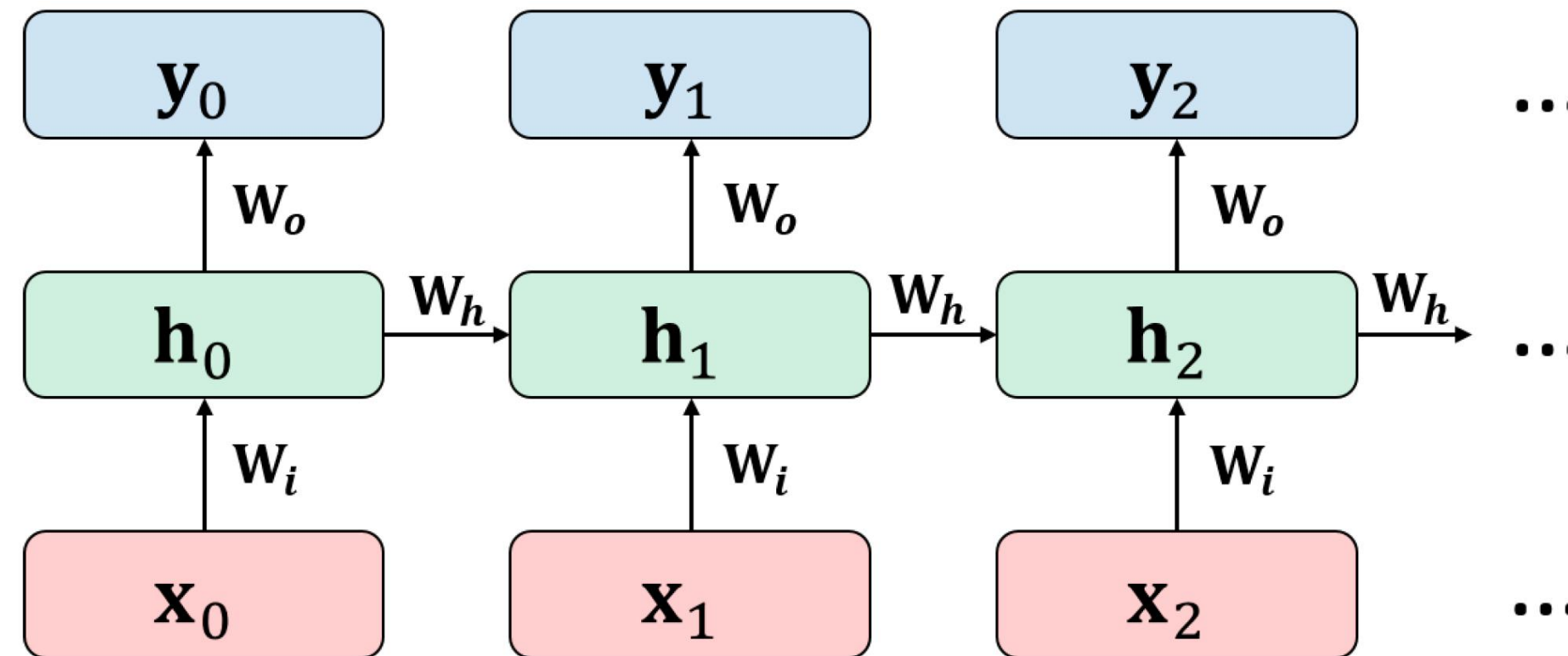
- **Keeps hidden states stable**
 - *tanh* squashes outputs to $[-1, 1]$
 - Prevents *exploding* or *vanishing* gradients
- **Supports positive & negative signals**
 - Outputs can be negative or positive
 - Helps model complex temporal dependencies
- **Zero-centered output improves learning**
 - Mean activation ≈ 0 , helps gradients flow better
 - Leads to faster and more stable convergence



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

Recurrent Neural Networks: Basics



$$\mathbf{h}_t = g(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t + \mathbf{b}_h)$$

$$\mathbf{y}_t = \sigma(\mathbf{W}_o \mathbf{h}_t + \mathbf{b}_o)$$

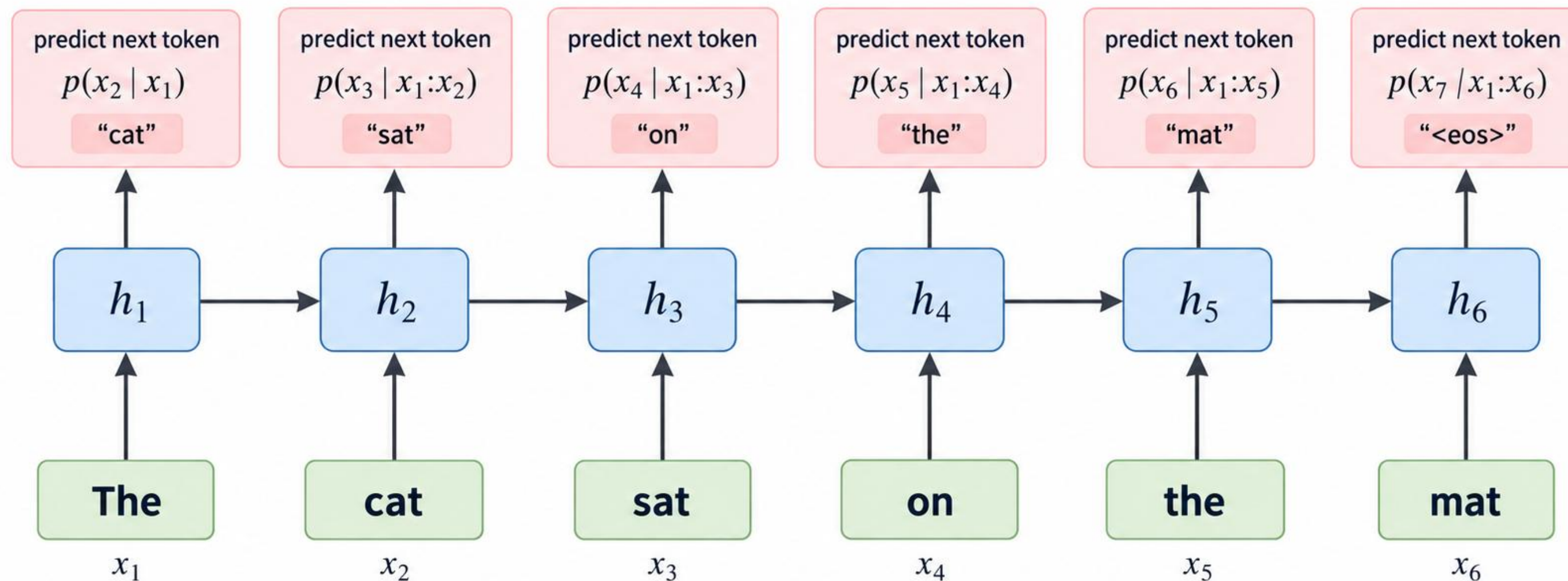
□ **Note:** There can be **an output at every time step!** The output of one time step can be the input for the next time step.

An RNN LM Predicts at Every Step

- Each hidden state h_t is projected to vocabulary logits.

- **Softmax** converts logits into a next-token distribution.

One sequence therefore supplies many next-token prediction problems.



Takeaway: At each step, the hidden state summarizes the history so far and is used to predict the next token.

Softmax

□ For a vector z with dimension k , **softmax** function is defined as:

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^k \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^k \exp(z_i)}, \dots, \frac{\exp(z_k)}{\sum_{i=1}^k \exp(z_i)} \right]$$

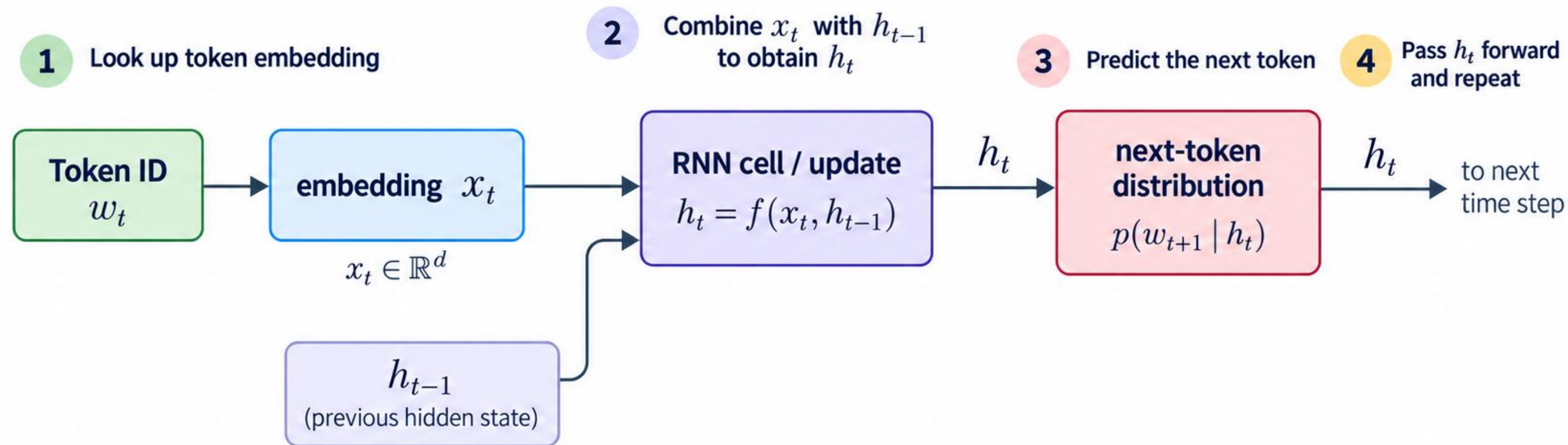
□ Example : $z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

The RNN Language-Model Pipeline

- Token ID $\rightarrow$ embedding x_t .
 - Combine x_t with h_{t-1} to obtain h_t .
- Use h_t to predict the next-token distribution.
Pass h_t forward and repeat.



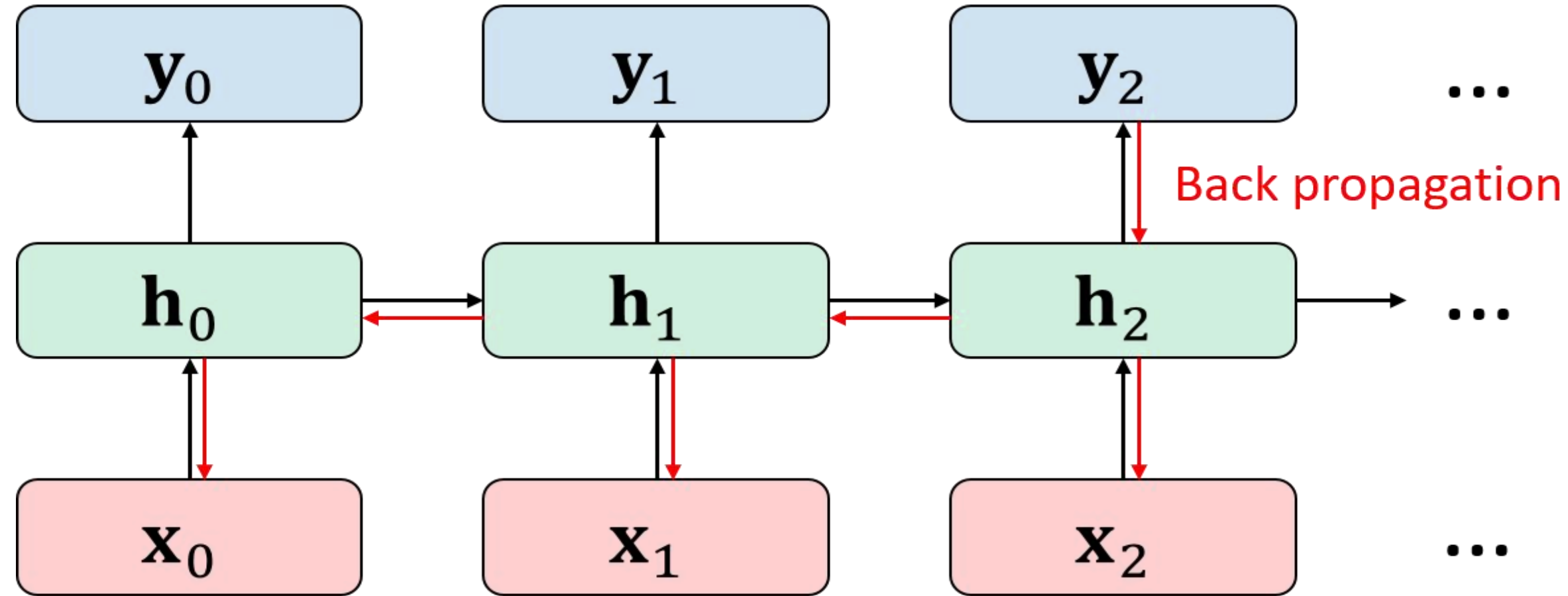
Takeaway: At each step, the model updates its hidden state and uses it to predict the next-token distribution.

Outline

- Language Modeling
- Recurrent Neural Networks
- **RNN LANGUAGE MODELING AND TRAINING**
- RNN Applications and Seq2seq
 - RNN Limitations and the Bridge to Attention
 - Practice & Discussion
 - Takeaways

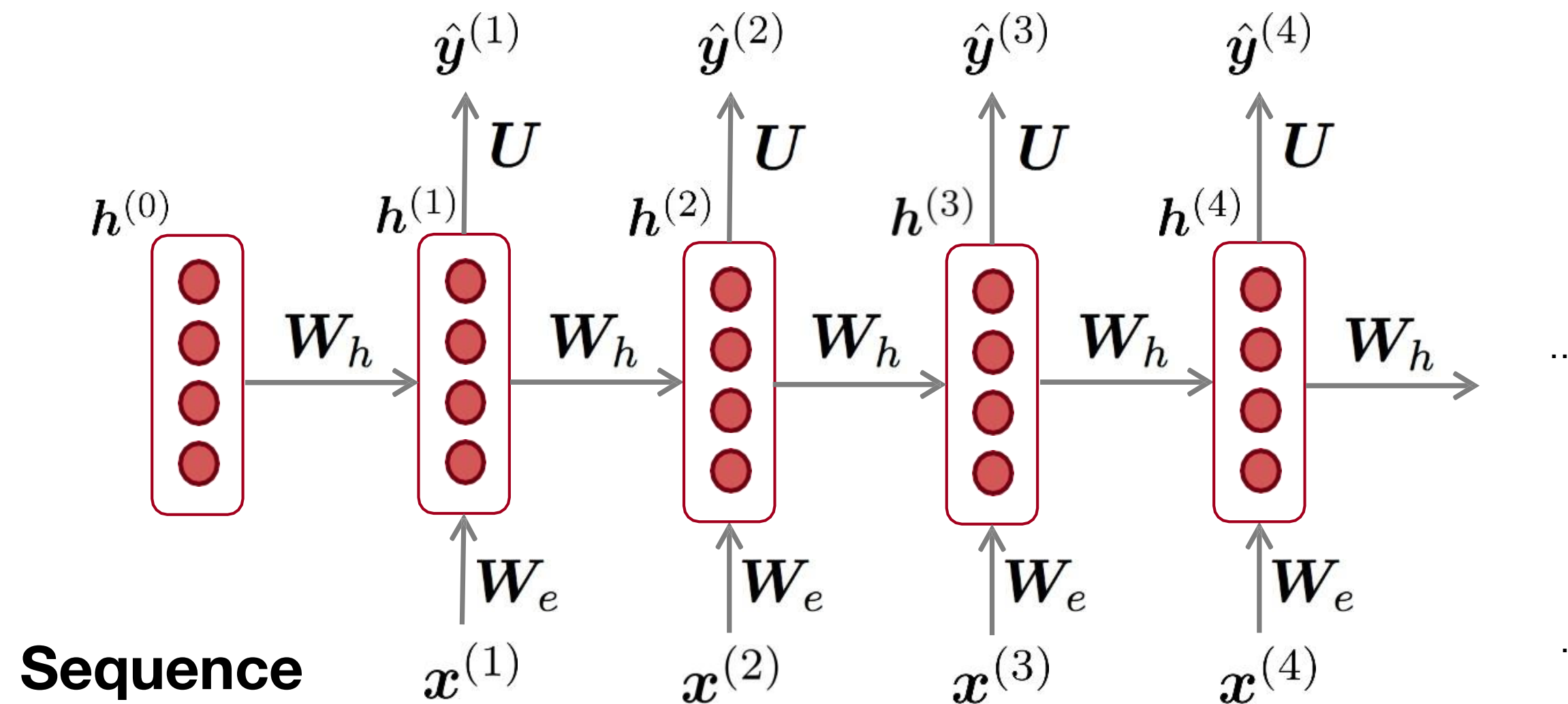
Recurrent Neural Networks: Training

- Forward computation uses shared parameter matrices.
- Backward computation updates the shared parameter matrices.



Recurrent Neural Networks: Training

- **Step 1:** Every input sequence $x^{(1)}, \dots, x^{(T)}$
- **Step 2:** Input each sequence word by word to RNN and calculate the output probability at each time step.



Recurrent Neural Networks: Training

- **Step 1:** Every input sequence $x^{(1)}, \dots, x^{(T)}$
- **Step 2:** Input each sequence word by word to RNN and calculate the output probability at each time step.
- **Step 3:** Use cross entropy to calculate the loss at each time step
 - Based on predicted distribution $\hat{y}^{(t)}$ and real word representation $y^{(t)}$.

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)}$$

Recurrent Neural Networks: Training

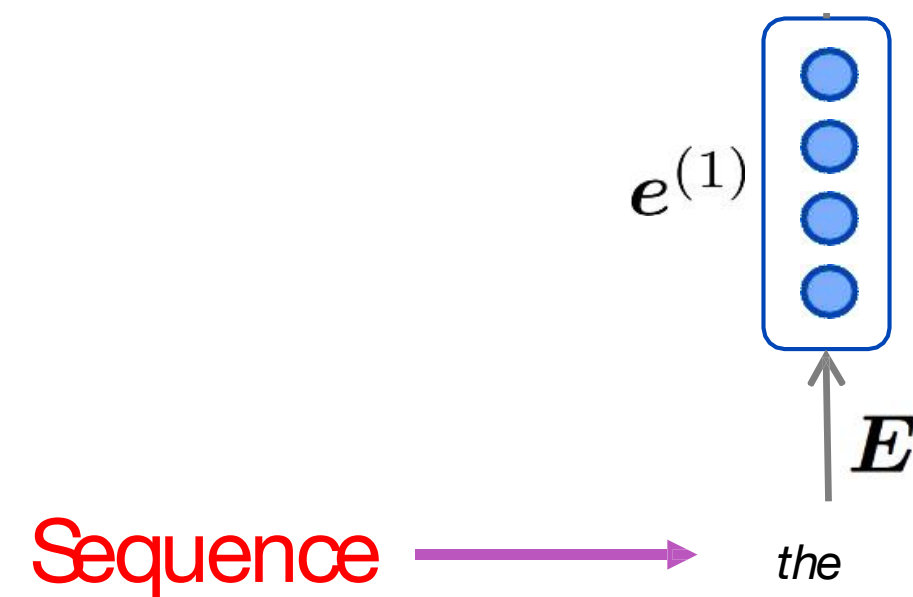
- **Step 1:** Every input sequence $x^{(1)}, \dots, x^{(T)}$
- **Step 2:** Input each sequence word by word to RNN and calculate the output probability at each time step.
- **Step 3:** Use cross entropy to calculate the loss at each time step
 - Based on predicted distribution $\hat{\mathbf{y}}^{(t)}$ and real word representation $\mathbf{y}^{(t)}$.

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)}$$

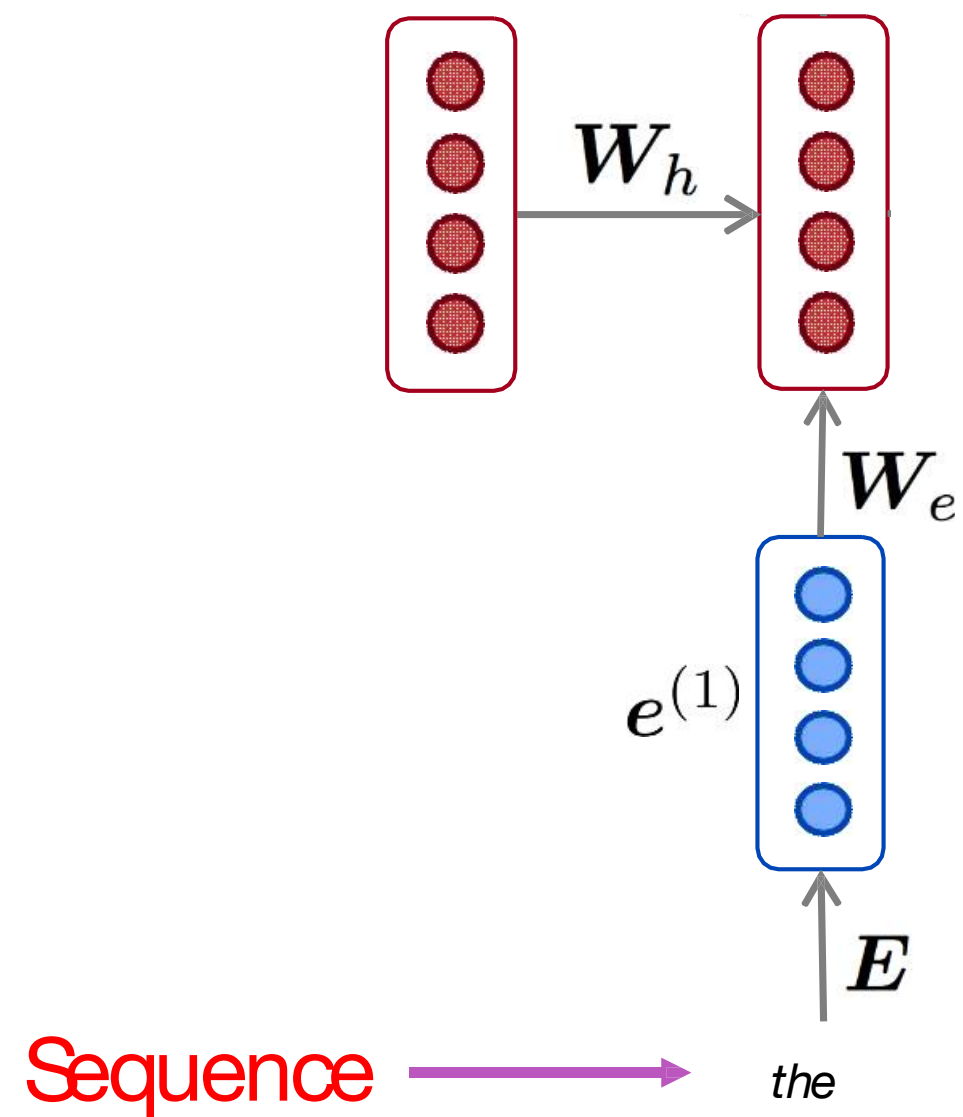
- **Step 4:** calculate the average loss of all time steps:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

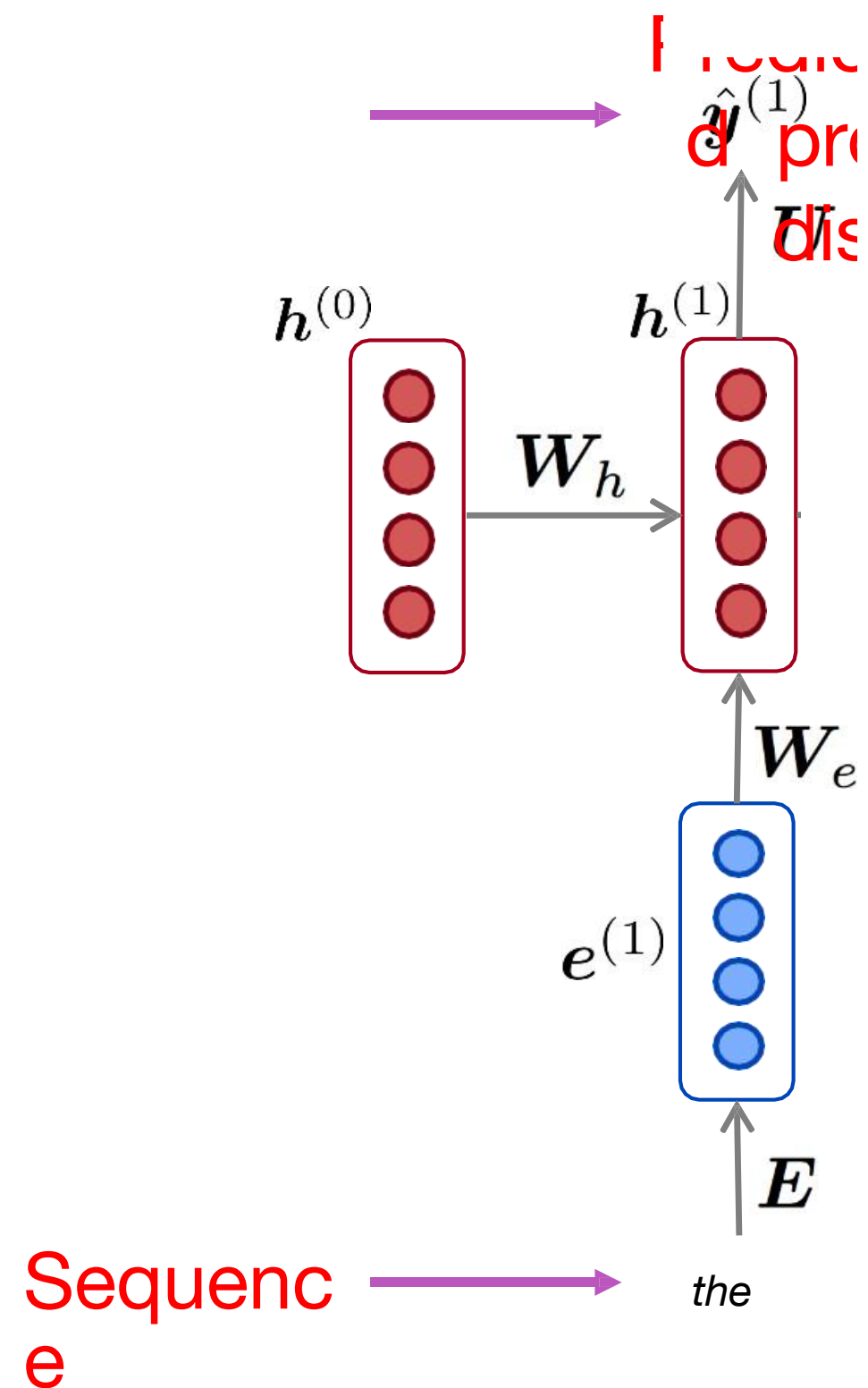
Recurrent Neural Networks: Training



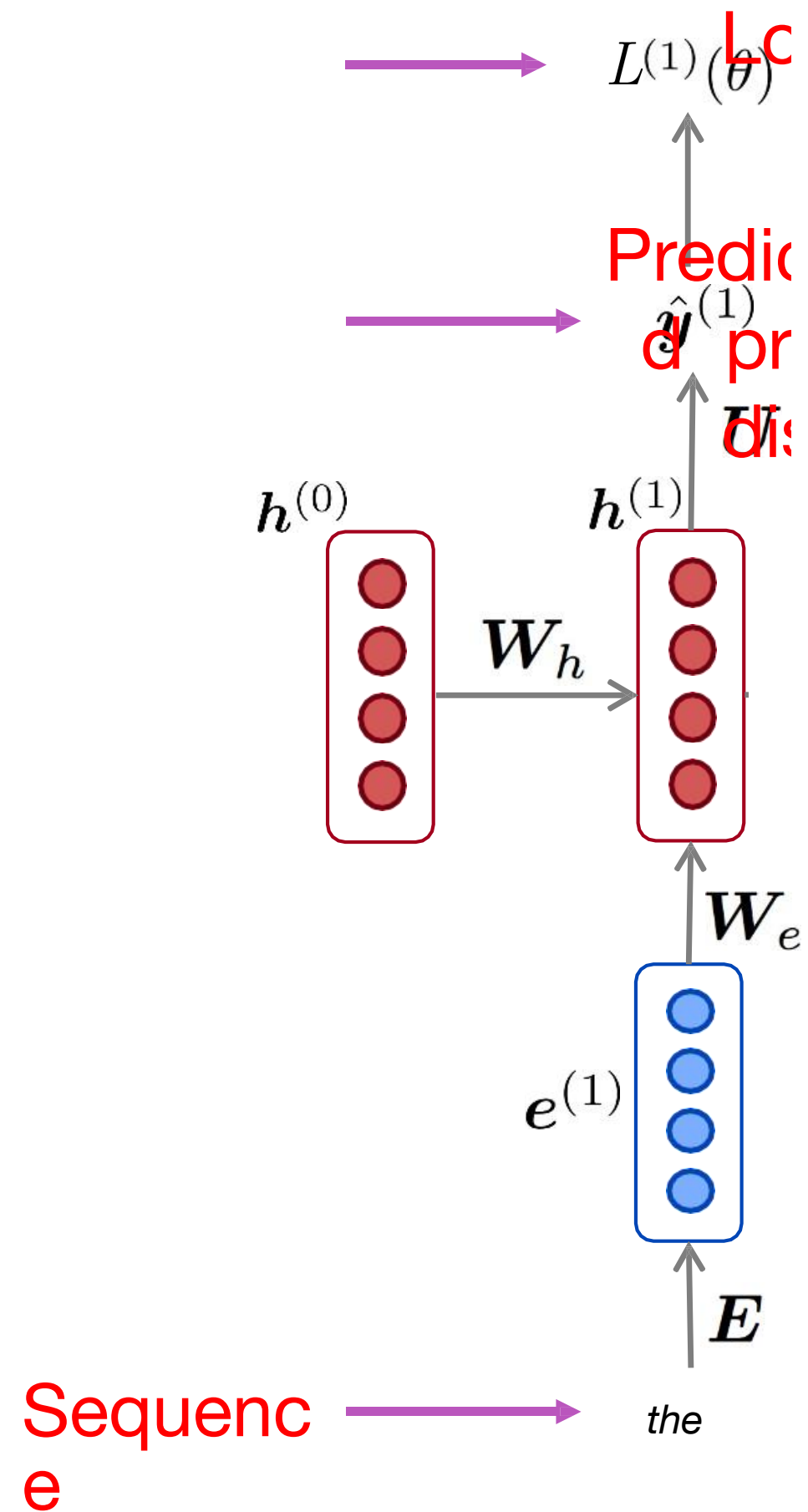
Recurrent Neural Networks: Training



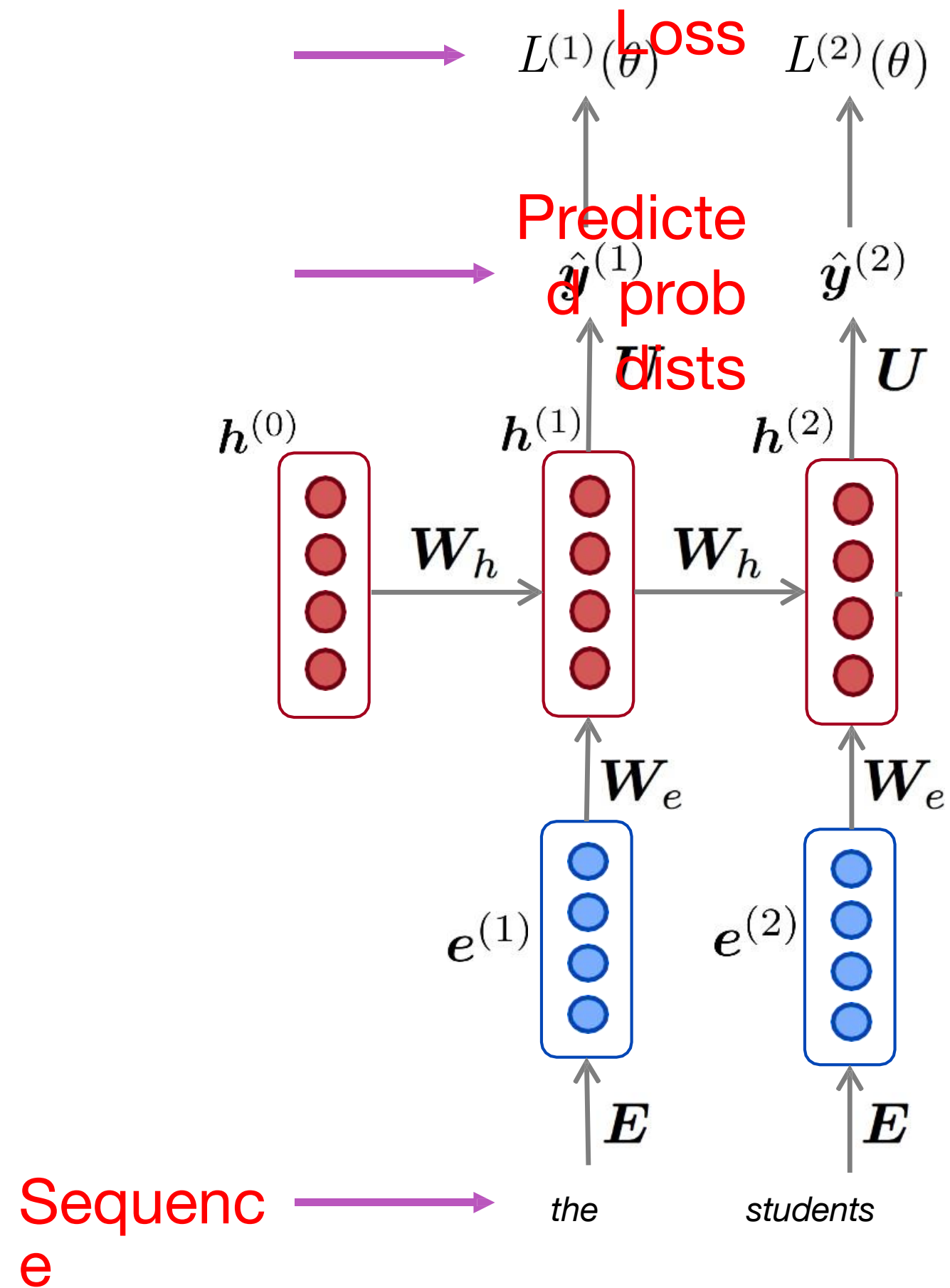
Recurrent Neural Networks: Training



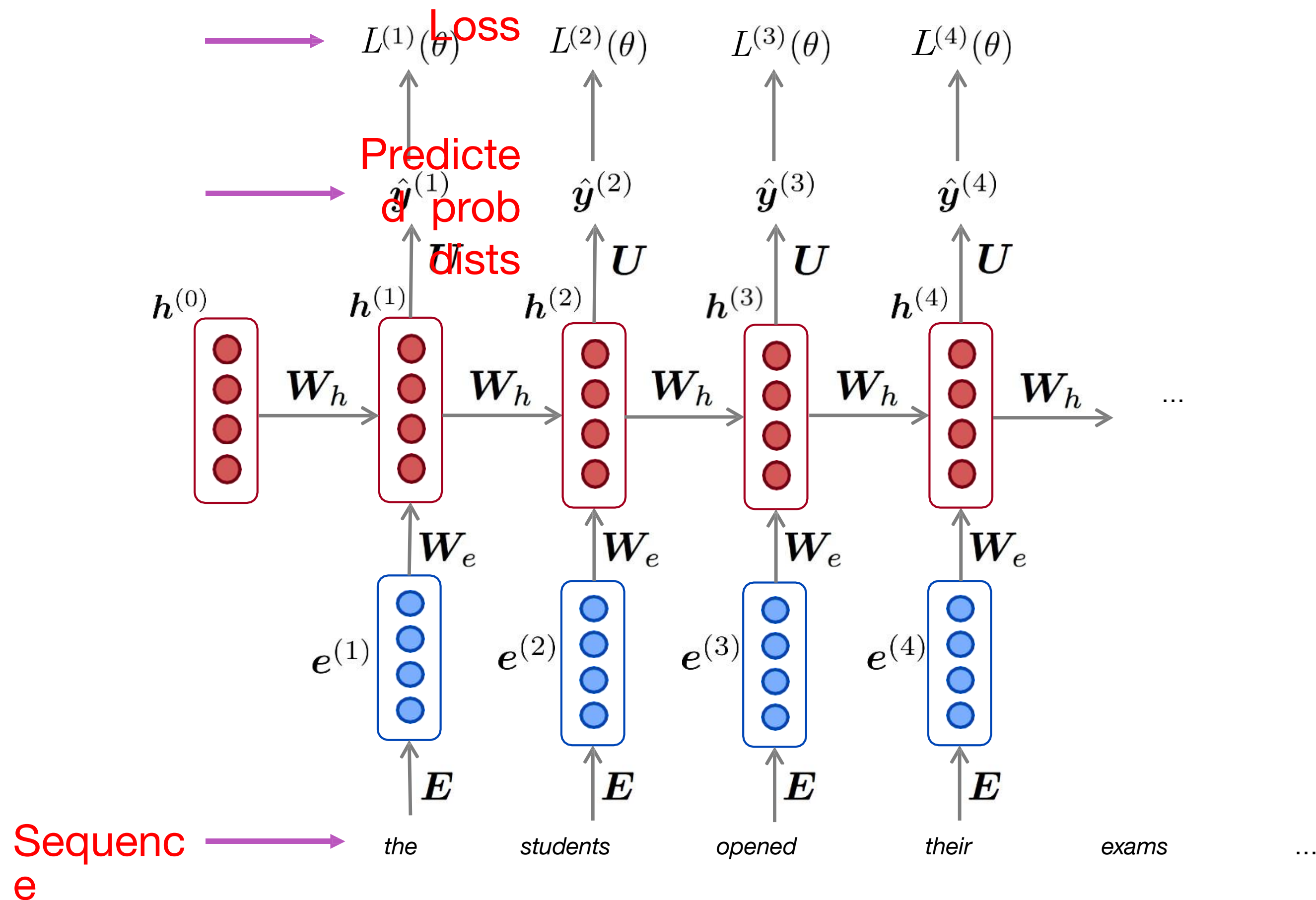
Recurrent Neural Networks: Training



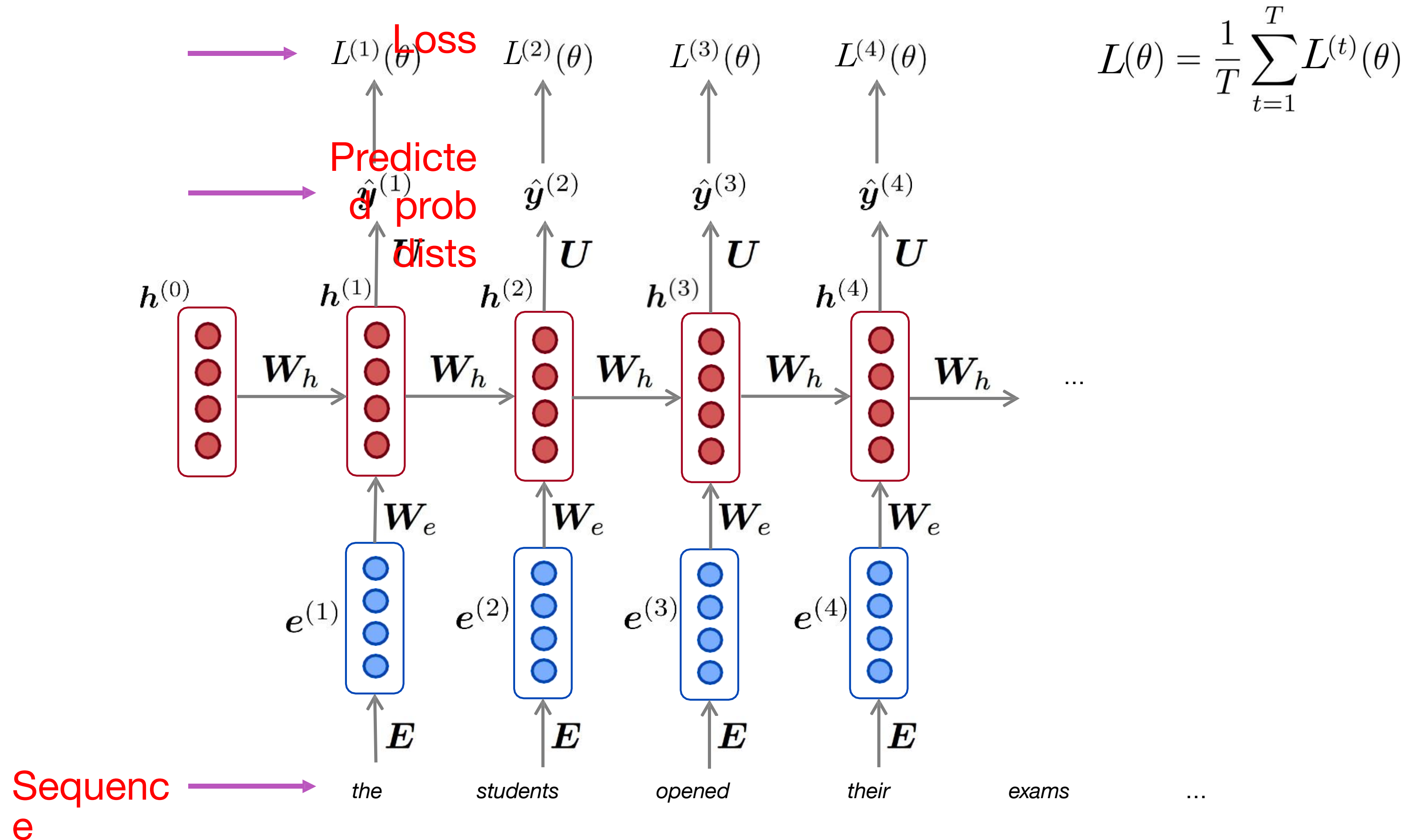
Recurrent Neural Networks: Training



Recurrent Neural Networks: Training



Recurrent Neural Networks: Training



Recurrent Neural Networks: Training

□ Consider a simplified RNN formula:

$$\begin{aligned}\mathbf{h}_t &= \mathbf{W} g(\mathbf{h}_{t-1}) + \mathbf{W}_i \mathbf{x}_t \\ \mathbf{y}_t &= \sigma(\mathbf{W}_o \mathbf{h}_t)\end{aligned}$$

□ Sum of losses over all time steps:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{t=1}^T \frac{\partial L_t}{\partial \mathbf{W}}$$

□ Chain rule:

$$\frac{\partial L_t}{\partial \mathbf{W}} = \sum_{k=1}^t \frac{\partial L_t}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}}$$

Recurrent Neural Networks: Training

□ Continuing with the chain rule: $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$

This can lead to **gradient exploding** or **vanishing gradient**.
(梯度爆炸) (梯度消失)

□ To mitigate exploding gradients, **gradient clipping** can be used:

$$\begin{aligned} &\text{if } \|\mathbf{g}\| > v, \\ &\text{then } \mathbf{g} \leftarrow \frac{\mathbf{g}}{\|\mathbf{g}\|} v, \text{ where } v \text{ is a threshold} \end{aligned}$$

Recurrent Neural Networks: **Vanishing Gradient**

- The **Vanishing Gradient** phenomenon leads to problems:
 - The prediction error calculated at time step t **cannot be effectively propagated back** to the parameters at an earlier time step k , especially when k is far from the current time step t .
 - *Vanishing gradients* make it difficult for RNNs to learn and capture **long-term dependencies** between distant time steps in a sequence.

Solutions for Vanishing Gradients: **LSTM/GRU**

- ❑ **Long short-term memory (LSTM)** as a variant of an RNN can alleviate the **vanishing gradient** problem:
 - **Short-term memory**: saves a small amount of information.
 - **Long-term memory**: information can be preserved for a longer time.
- ❑ **Gated Recurrent Unit (GRU)** is a simplified version of LSTM, which also has memory and forgetting mechanisms.

These RNN variants are conceptually similar but differ in their computational units.

Outline

- Language Modeling
- Recurrent Neural Networks
- Rnn Language Modeling And Training
- **RNN APPLICATIONS AND SEQ₂SEQ**

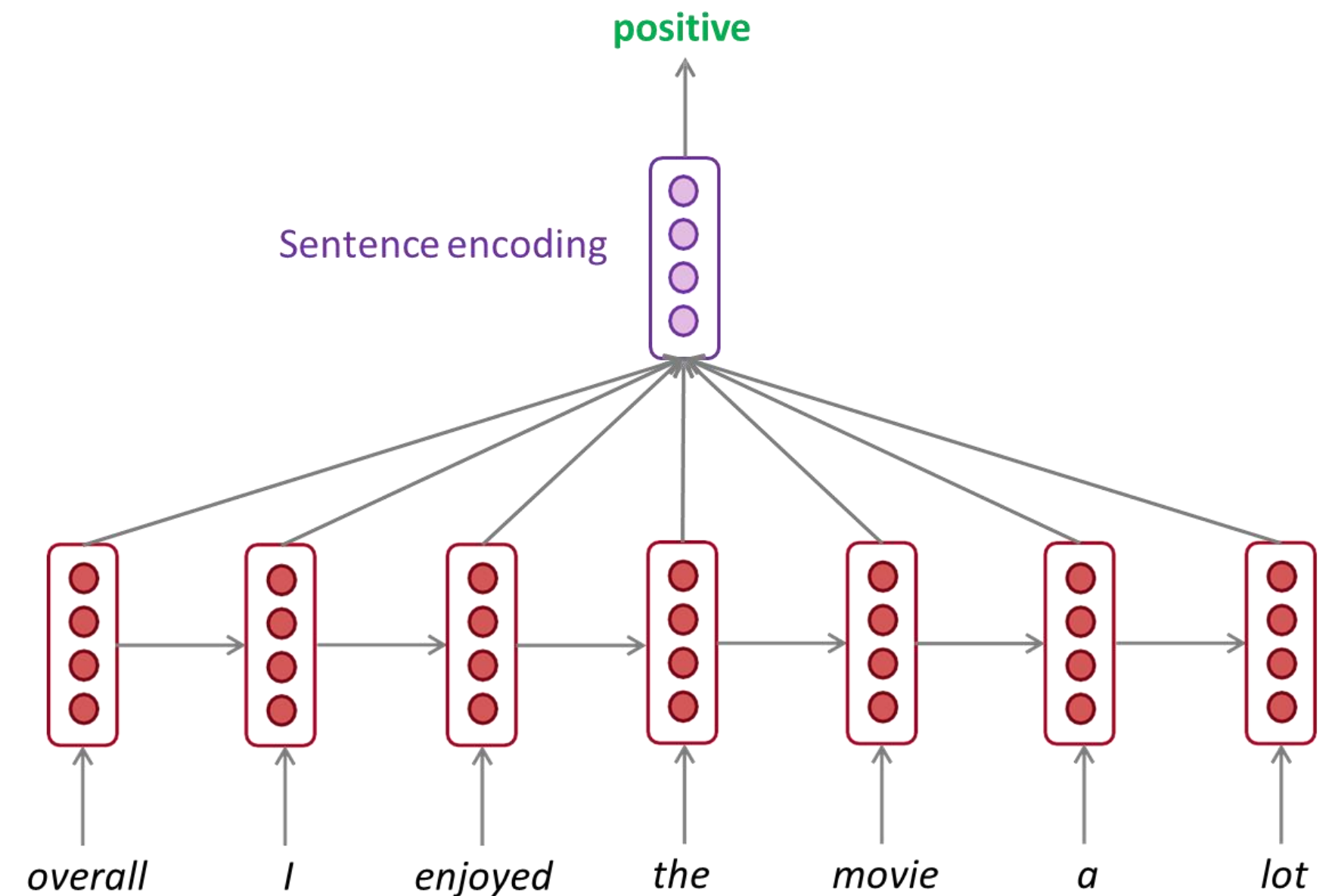
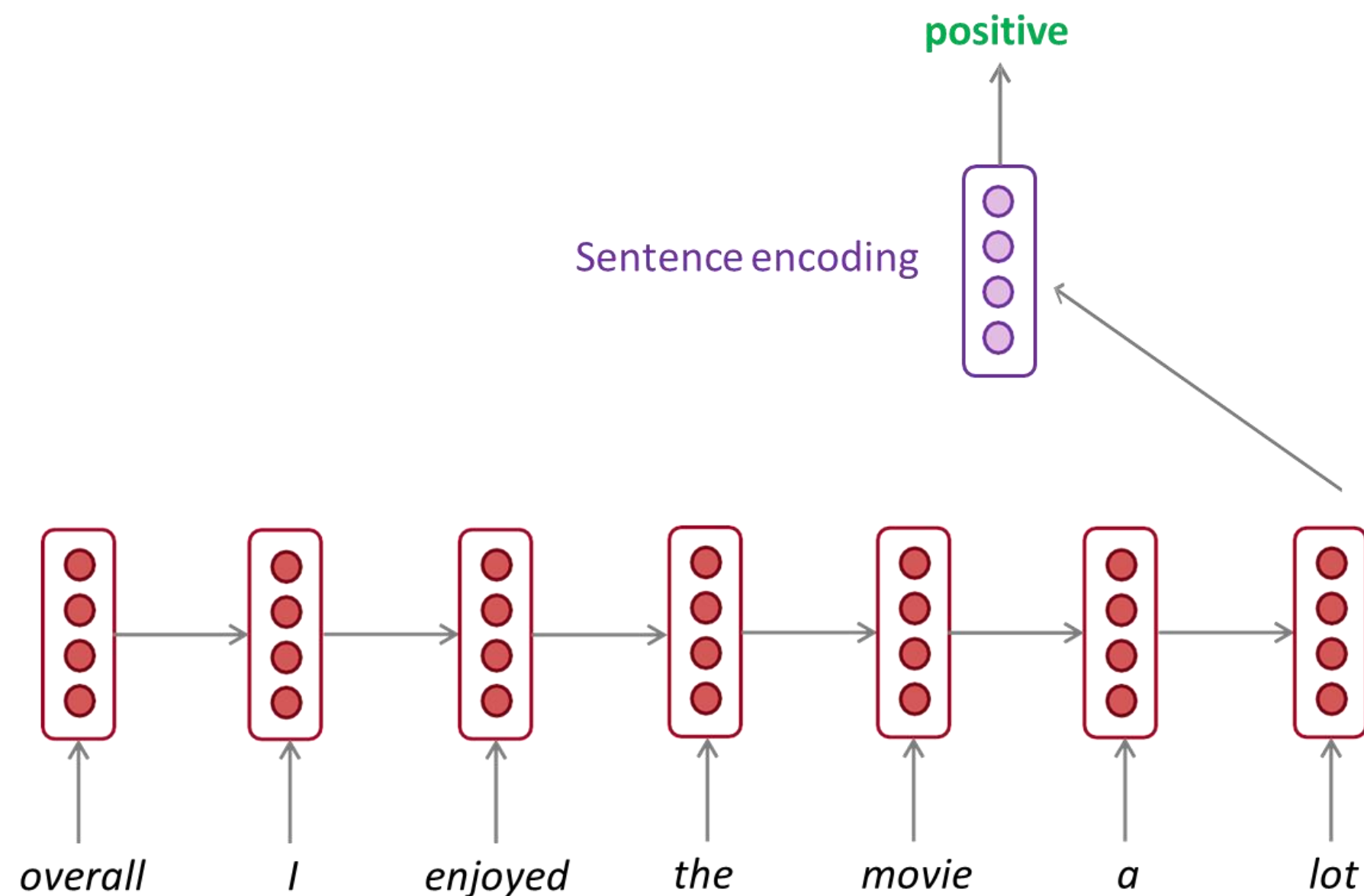
RNN Limitations and the Bridge to Attention

Practice & Discussion

Takeaways

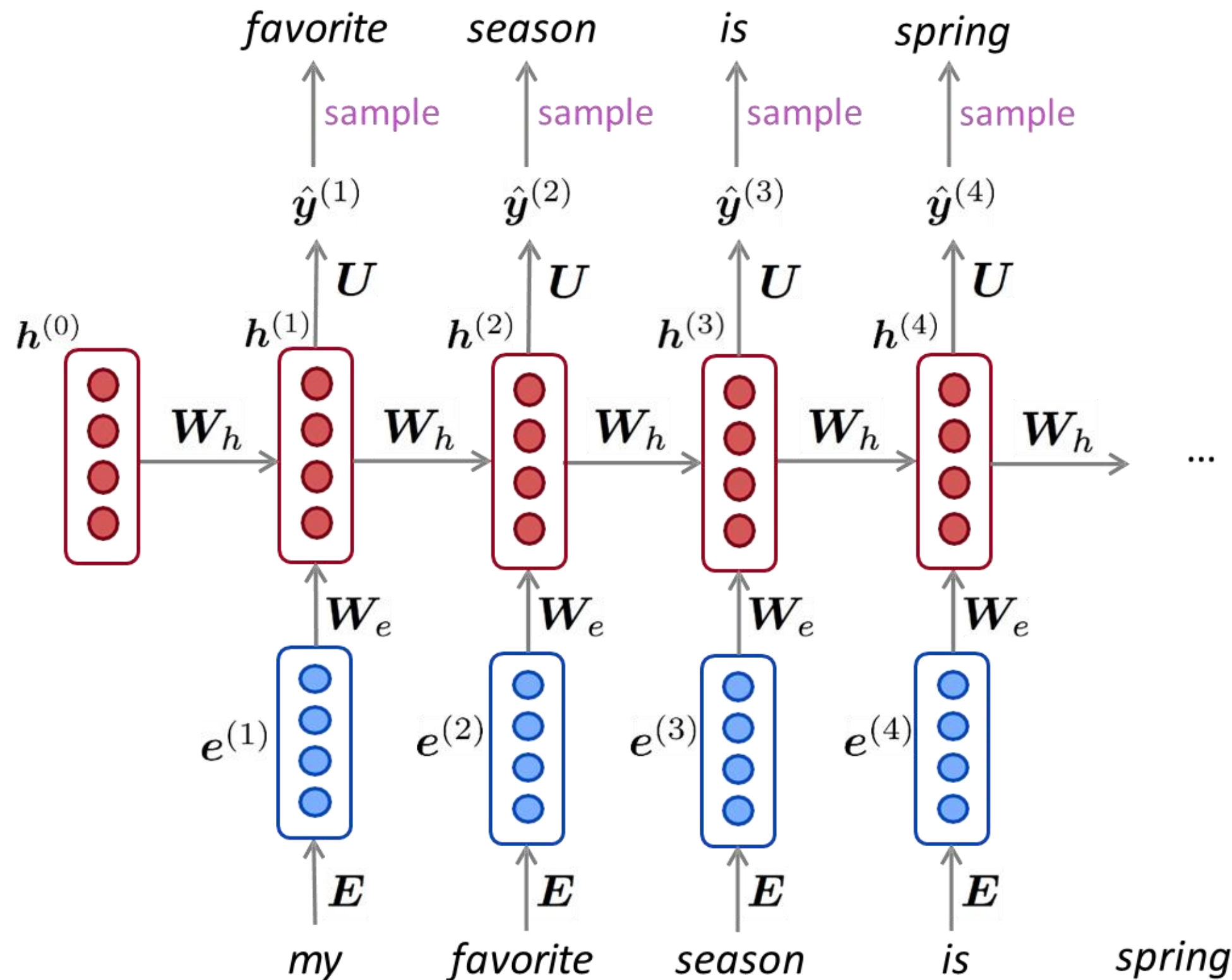
RNN Applications: Text Classification

- RNN can be used for text classification.
- How to predict the class?



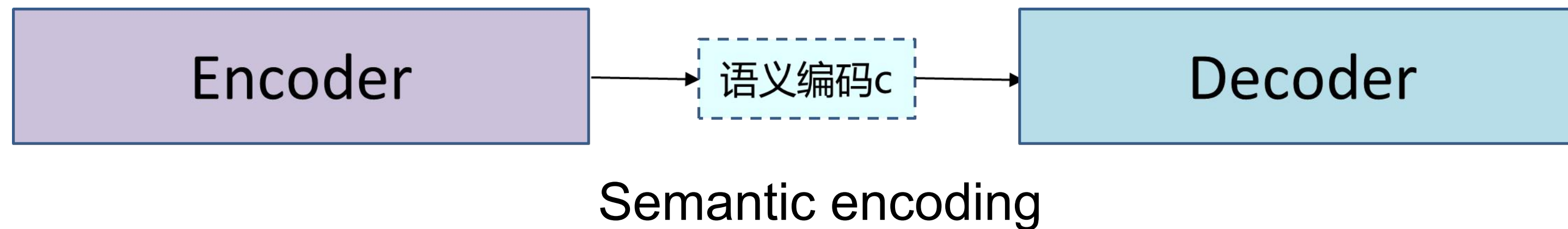
RNN Applications: Text Generation

- RNN can generate text through recurrent sampling.



RNN Applications: Neural Machine Translation

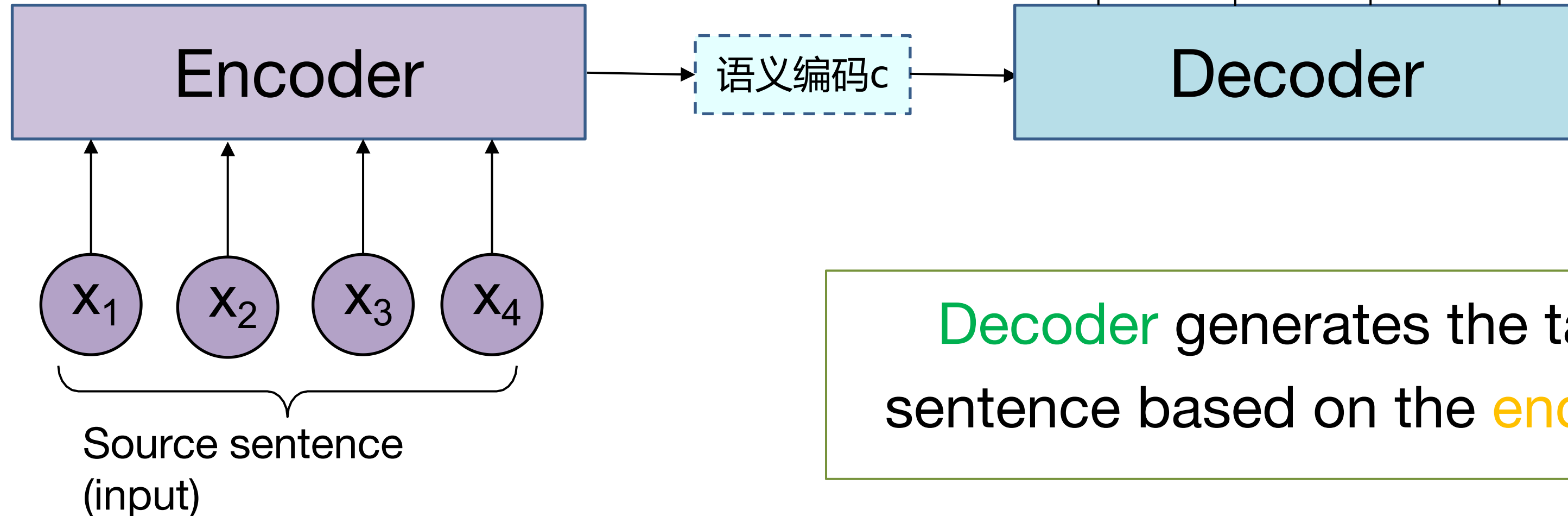
- Modeling machine translation with neural networks.
 - The **encoder** is used for understanding the source language.
 - The **decoder** is used for generating the target language.



RNN Applications: Neural Machine Translation

seq2seq model

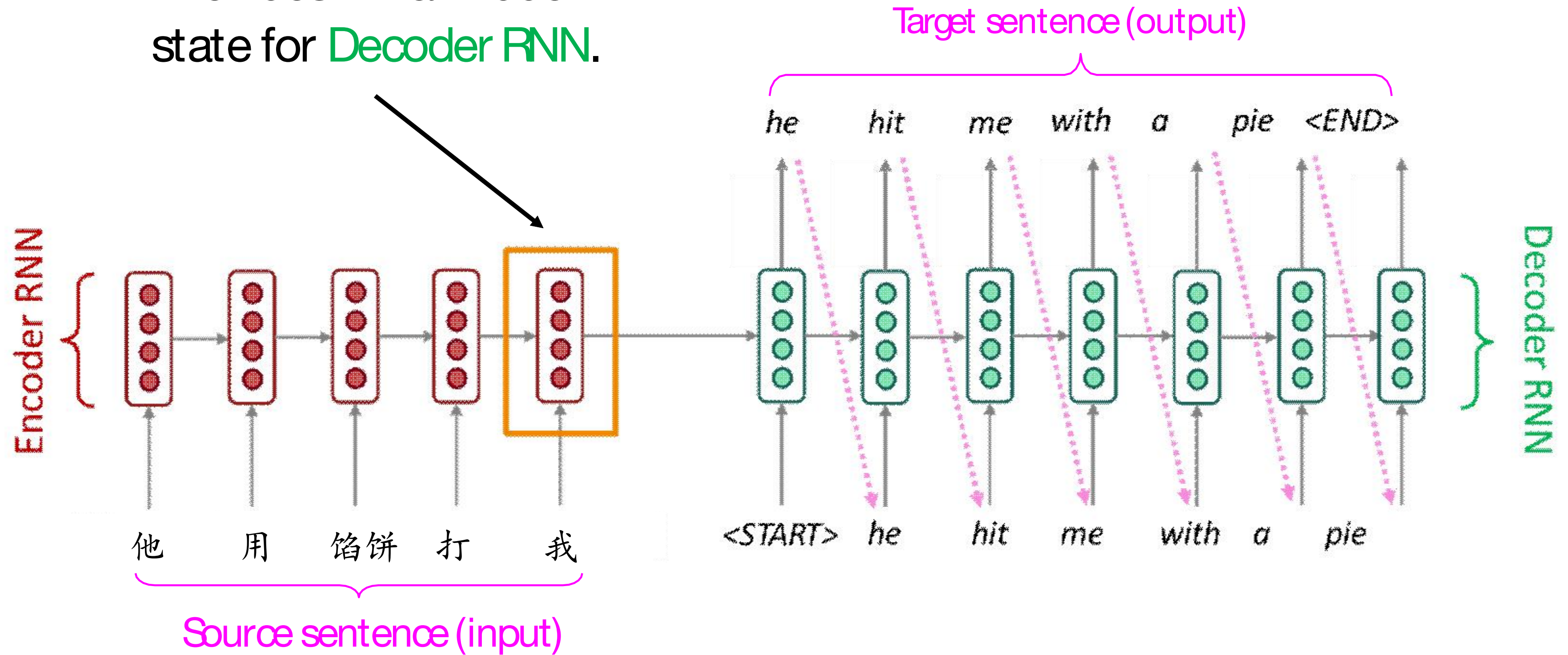
Encoder generates an encoding of the input source sentence.



Decoder generates the target sentence based on the encoding.

Neural Machine Translation (NMT)

Provides initial hidden state for Decoder RNN.



Neural Machine Translation (NMT)

- The seq2seq model is a **Conditional Language Model**.
 - **Language model**: The decoder predicts the next word of the target sentence y .
 - **Conditional**: Its prediction also depends on the source sentence x .
- NMT directly computes $P(y|x)$

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots P(y_T|y_1, \dots, y_{T-1}, x)$$

the probability of the next target word, given the currently generated target words and input

Question: How to train NMT?

Answer: Get a big parallel corpus...

Neural Machine Translation (NMT)

Parallel Corpus

他 用 馅饼 打 我

Source sentence (from corpus)

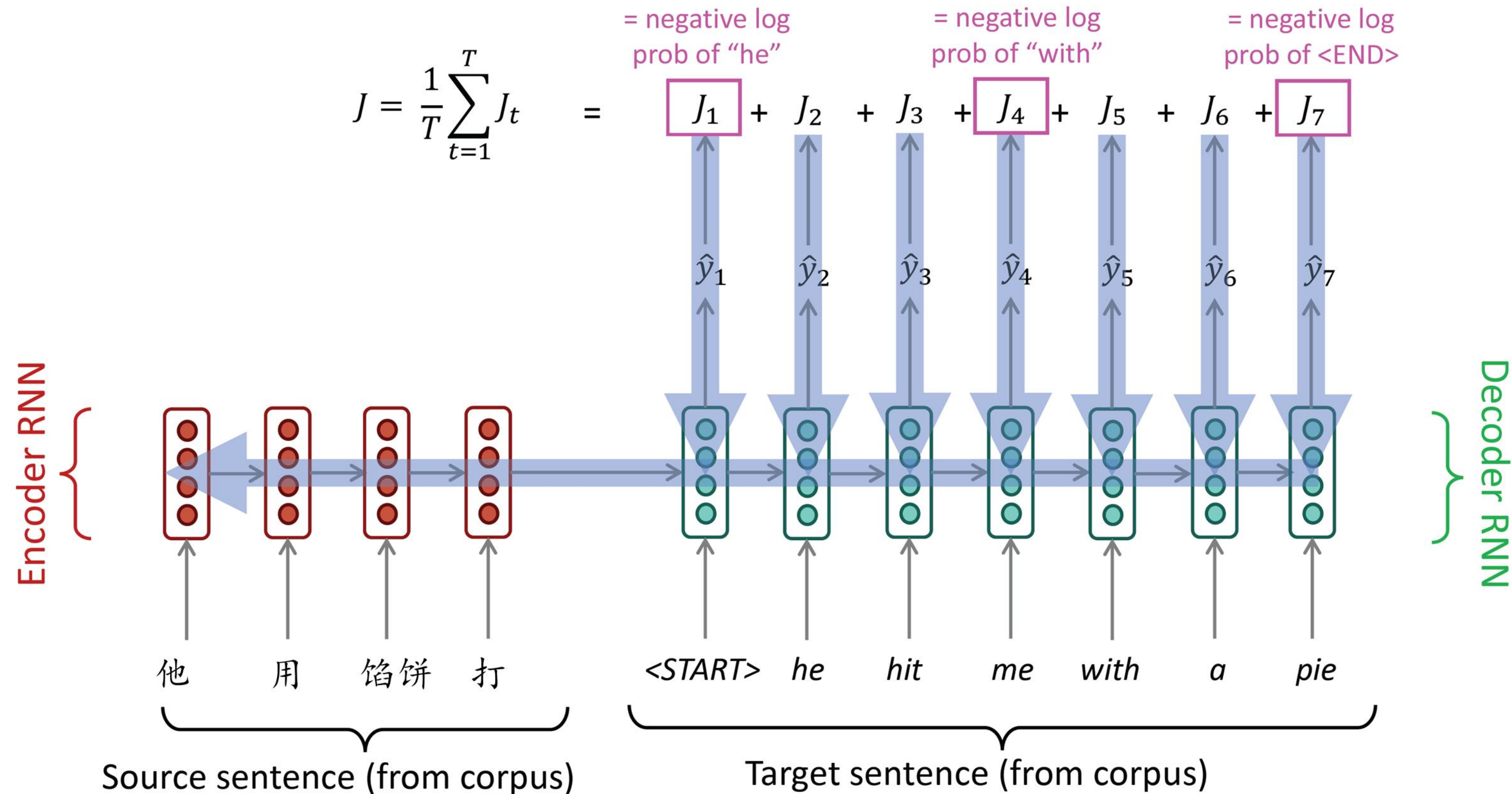
⋮

he hit me with a pie

Target sentence (from corpus)

⋮

NMT Training

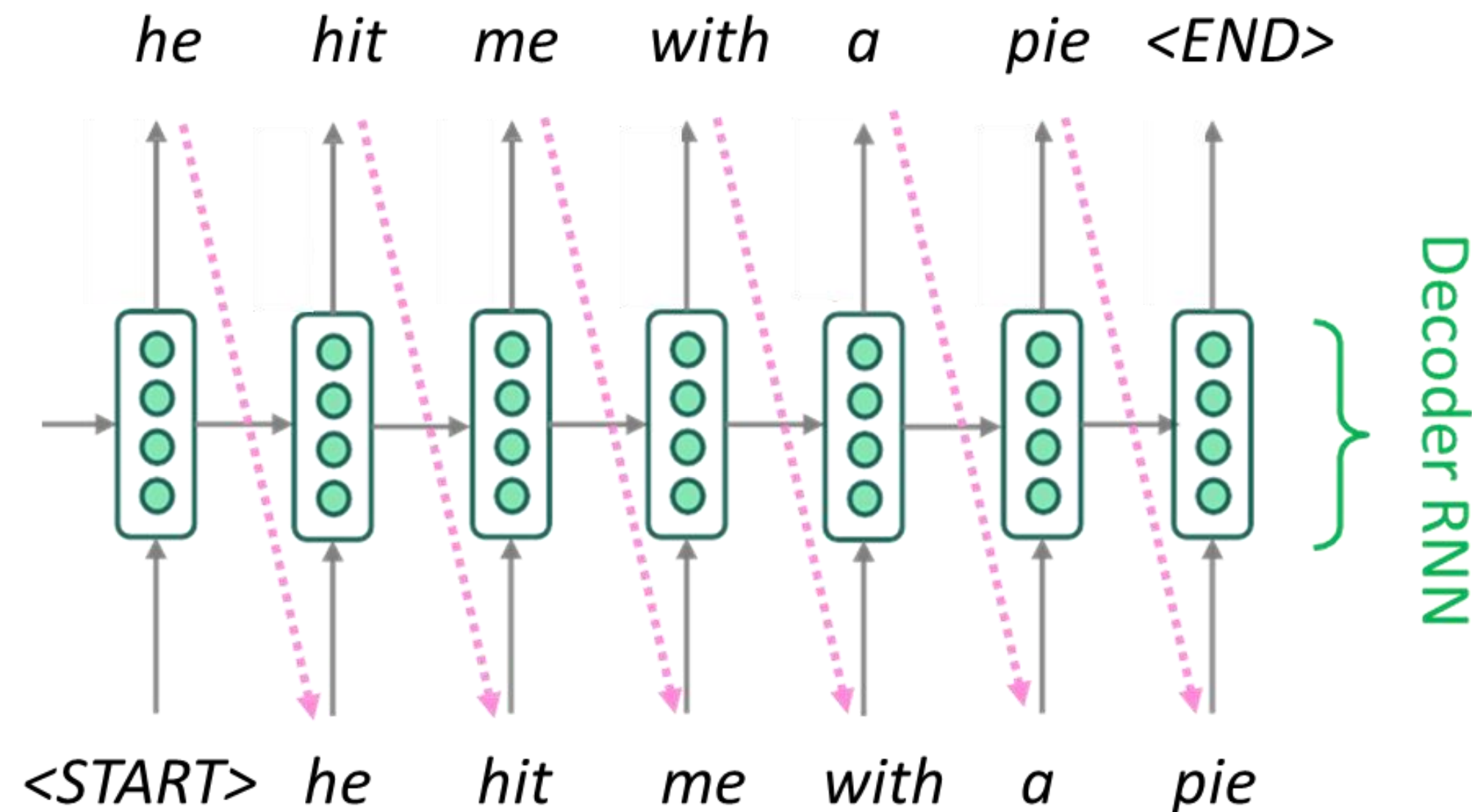


The Seq2seq model is optimized as a single system.
Backpropagation runs in an "end-to-end" manner.

NMT Training: Free-running Mode

□ Free-running mode

- **Training phase:** Use the model's predicted word as input for the next step.
- **Testing phase:** Pass the model-generated word to the next time step.



NMT Training: Free-running Mode

□ Advantages of free-running mode

- Training-testing consistency: The training process is identical to the inference process. The model does not rely on idealized “correct answers”, avoiding *exposure bias* (曝光偏差).
- Enhanced model robustness: The model learns to continue generating based on its own output, allowing it to better adapt and recover even if a previous step was erroneous.

NMT Training: Free-running Mode

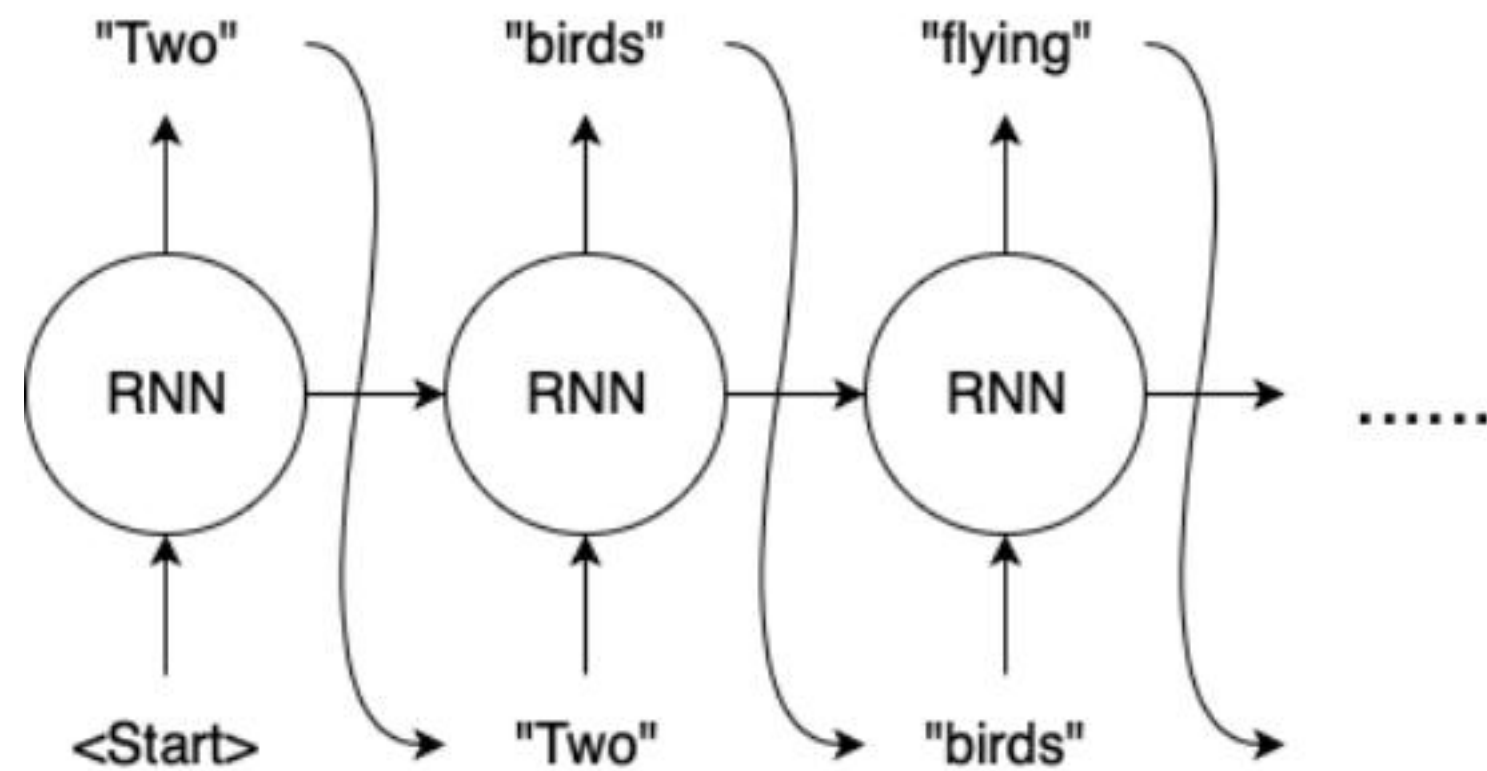
❑ Disadvantages of free-running mode

- **Unstable training, difficult to converge:** In the early stages, model predictions are often inaccurate. Feeding wrong words into the next step can cause the entire output to collapse in training.
- **Low training efficiency:** The model needs more time to explore and learn how to recover from imperfect inputs, which can slow down the training process or require more data.

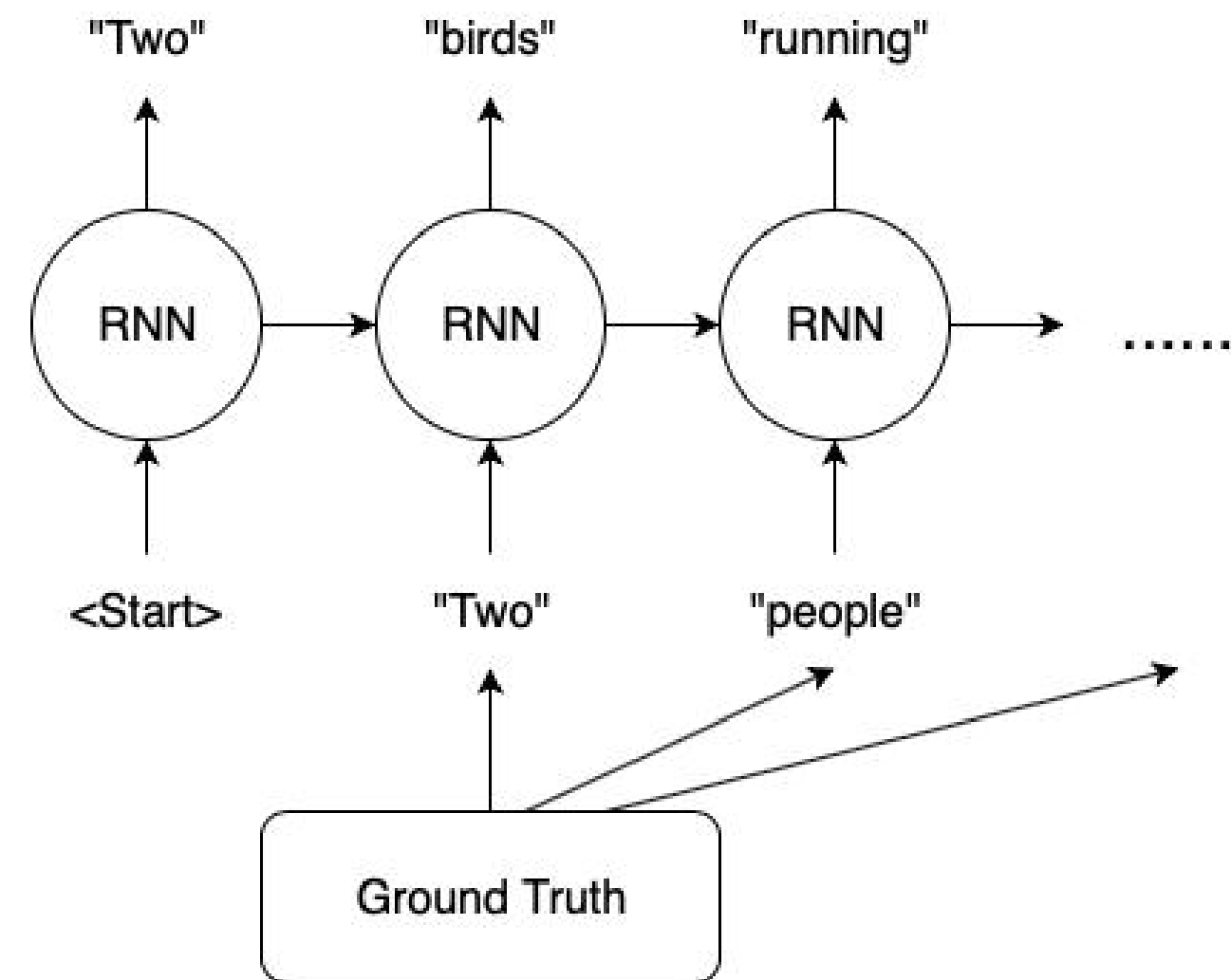
NMT Training: Teacher-forcing Mode

□ Teacher-forcing mode

- **Training:** Use **ground truth word** as input for each step of decoder.
- **Testing:** Use **model's predicted word** as input for the next time step.



Without Teacher Forcing



With Teacher Forcing

NMT Training: Teacher-forcing Mode

□ Advantages of teacher-forcing mode

- **Faster convergence:** Using ground truth labels as input makes it easier for the model to learn the correct output sequence, leading to faster training and reduced error propagation.
- **Avoids early training chaos:** When the model has not yet learned to predict effectively, its own output can be completely wrong. Using this wrong output as the next input would cause training chaos or collapse.

NMT Training: Teacher-forcing Mode

❑ Disadvantages of teacher-forcing mode

- **Exposure Bias**: During training, the model sees perfect, real inputs, but at test time it must rely on its own outputs. This inconsistency can lead to poor performance at test time, especially as errors accumulate.
- **Poor Long-Term Dependency**: Because it always gets the correct token next, the model doesn't learn to recover from mistakes. This leads to weaker ability to plan or maintain coherence over long sequences.

Outline

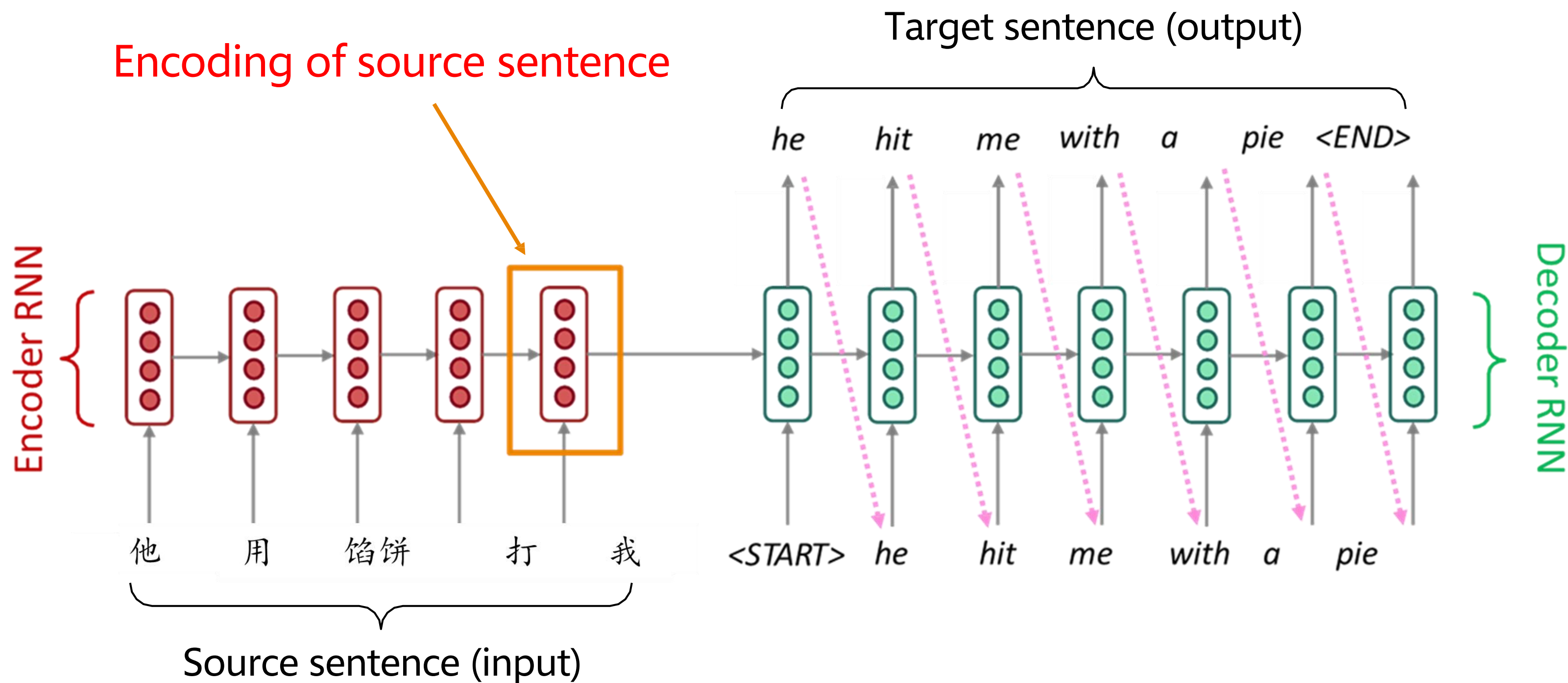
- Language Modeling
- Recurrent Neural Networks
- RNN Language Modeling And Training
- RNN Applications And Seq2seq

RNN LIMITATIONS AND THE BRIDGE TO ATTENTION

Practice & Discussion

Takeaways

Seq2seq: Bottleneck

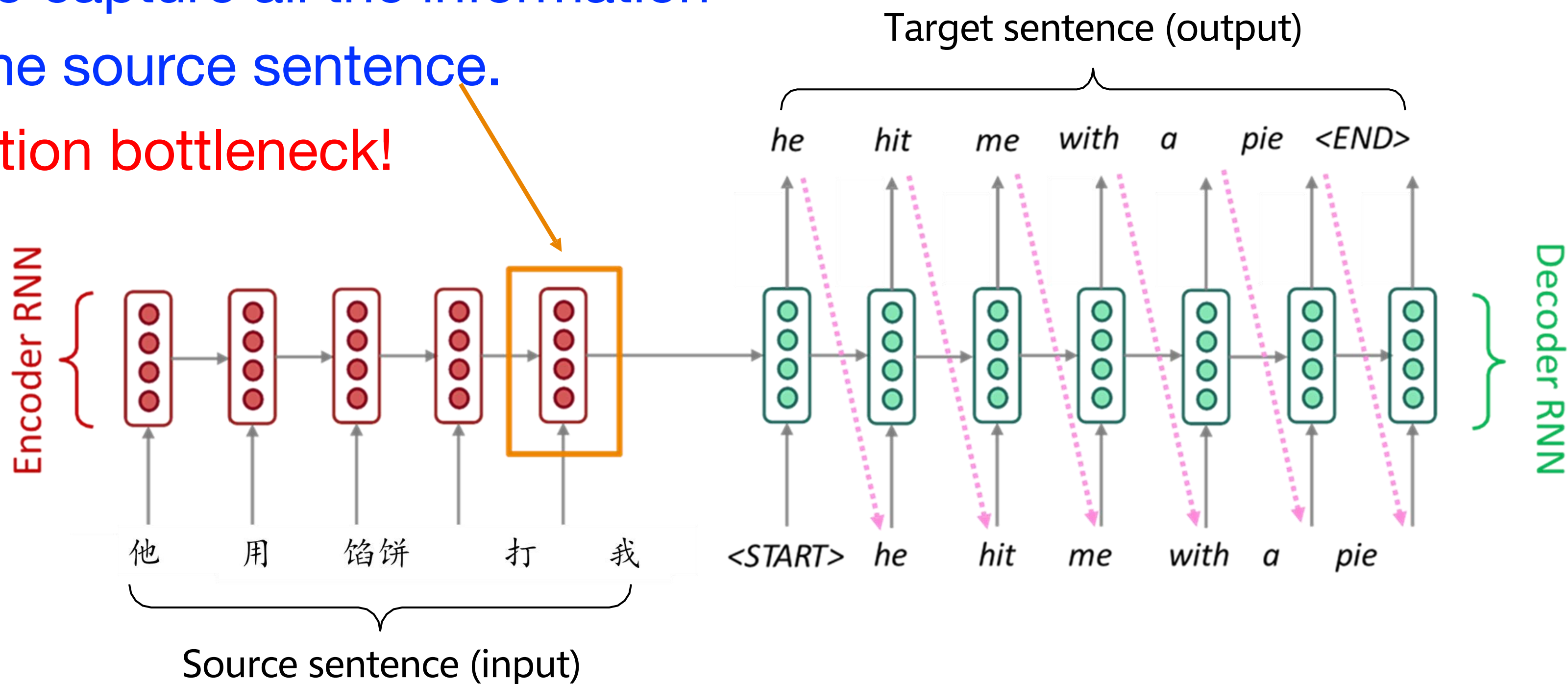


What is the problem with this architecture?

Seq2seq: Bottleneck

The encoding of the source sentence needs to capture all the information about the source sentence.

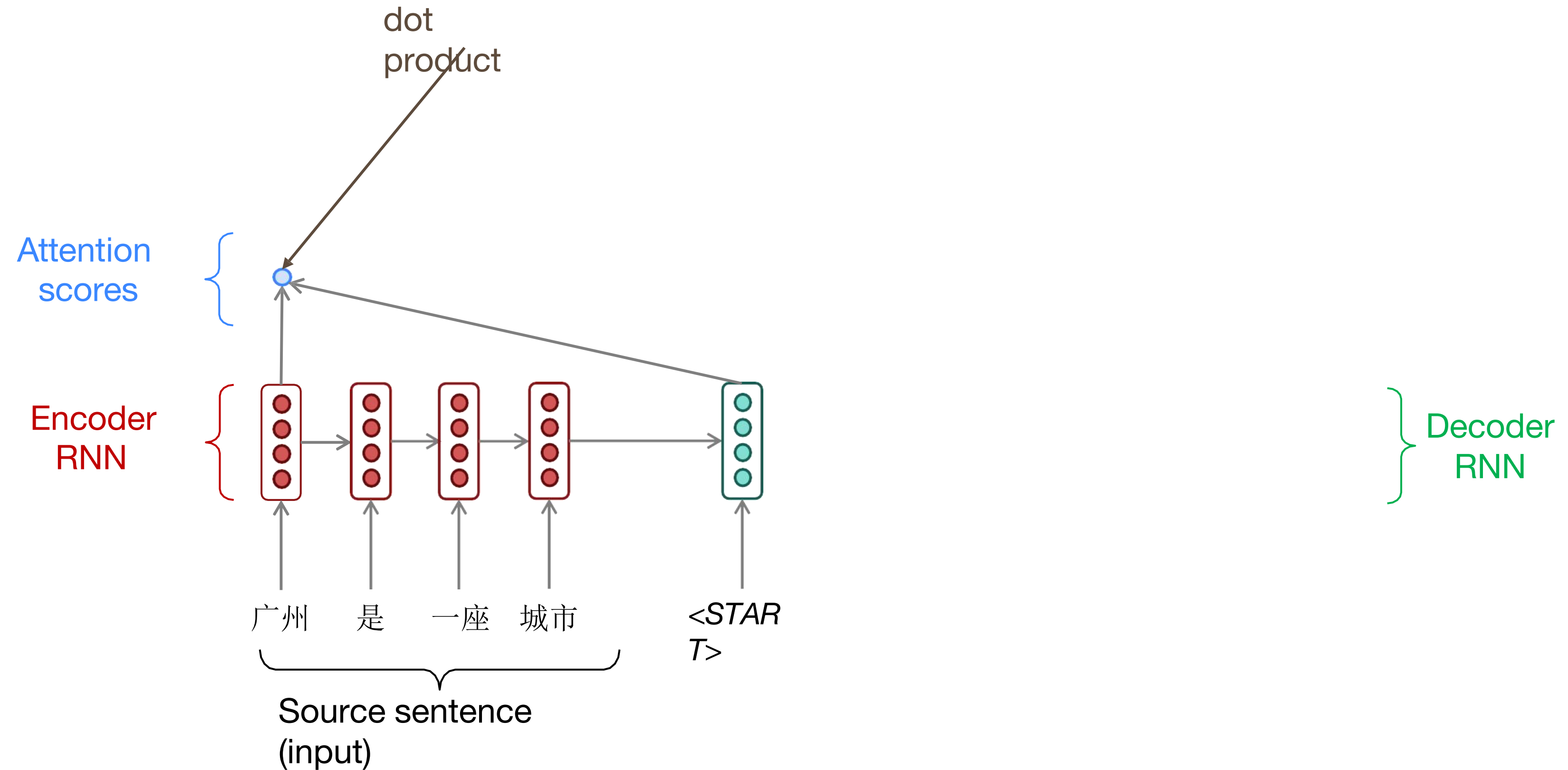
Information bottleneck!



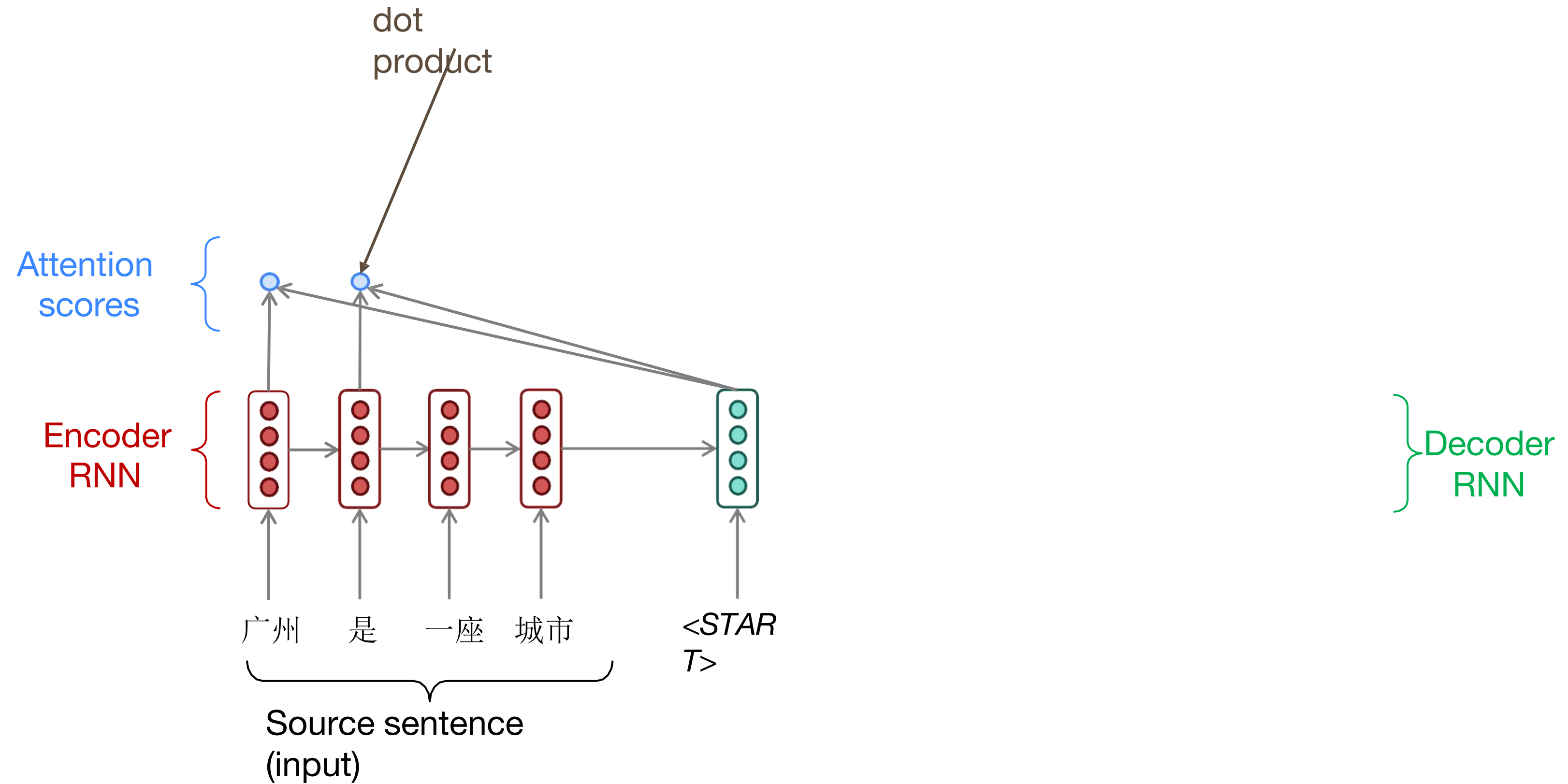
Seq2seq: **Attention**

- ❑ **Attention** provides a solution to the *bottleneck* problem.
- ❑ **Core idea:** At each step of the decoder, use a direct connection to the encoder to focus on specific parts of the source sequence.
- ❑ First, we'll show it with diagrams, then explain with formulas.

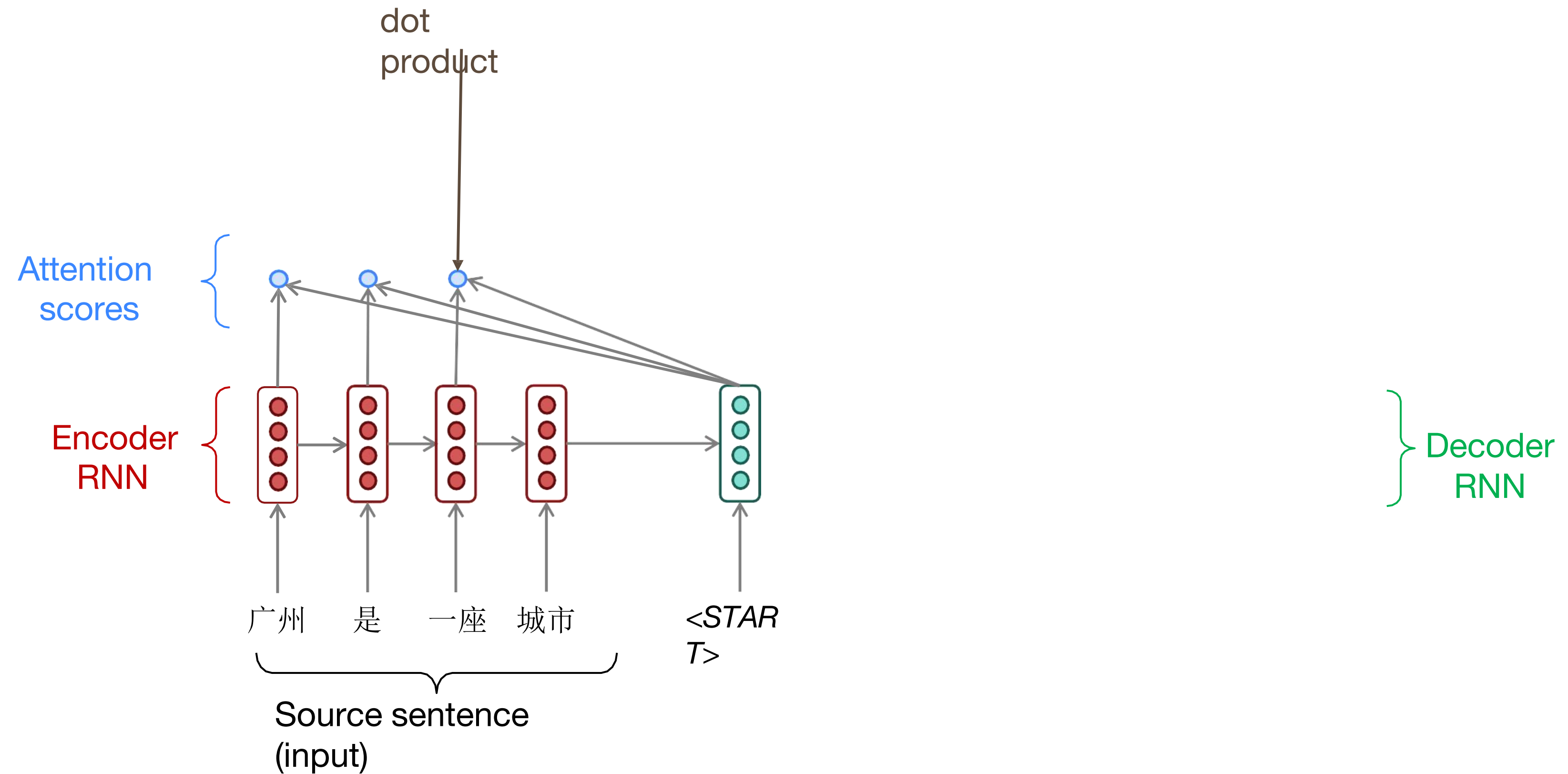
Seq2seq: Attention



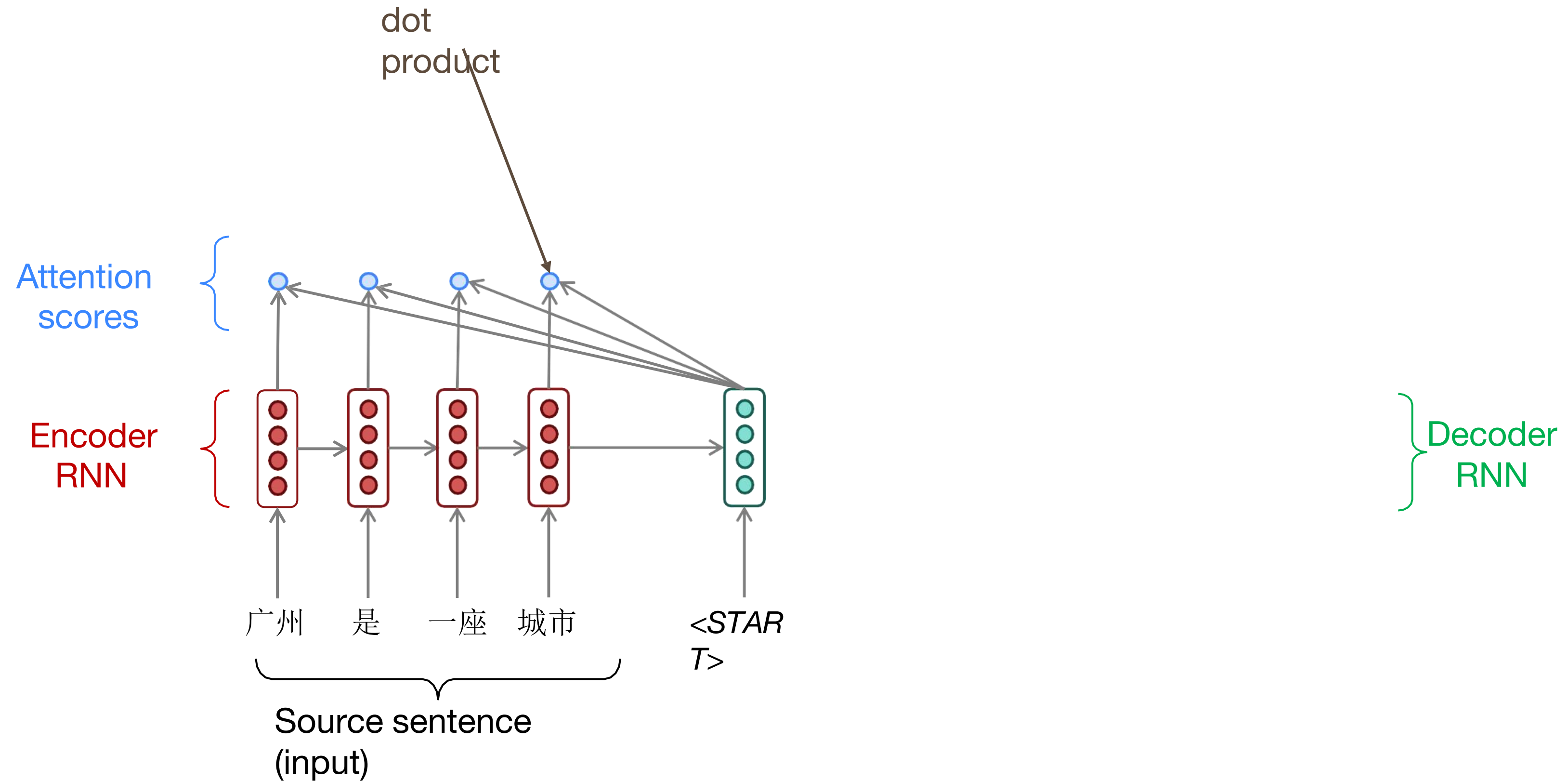
Seq2seq: Attention



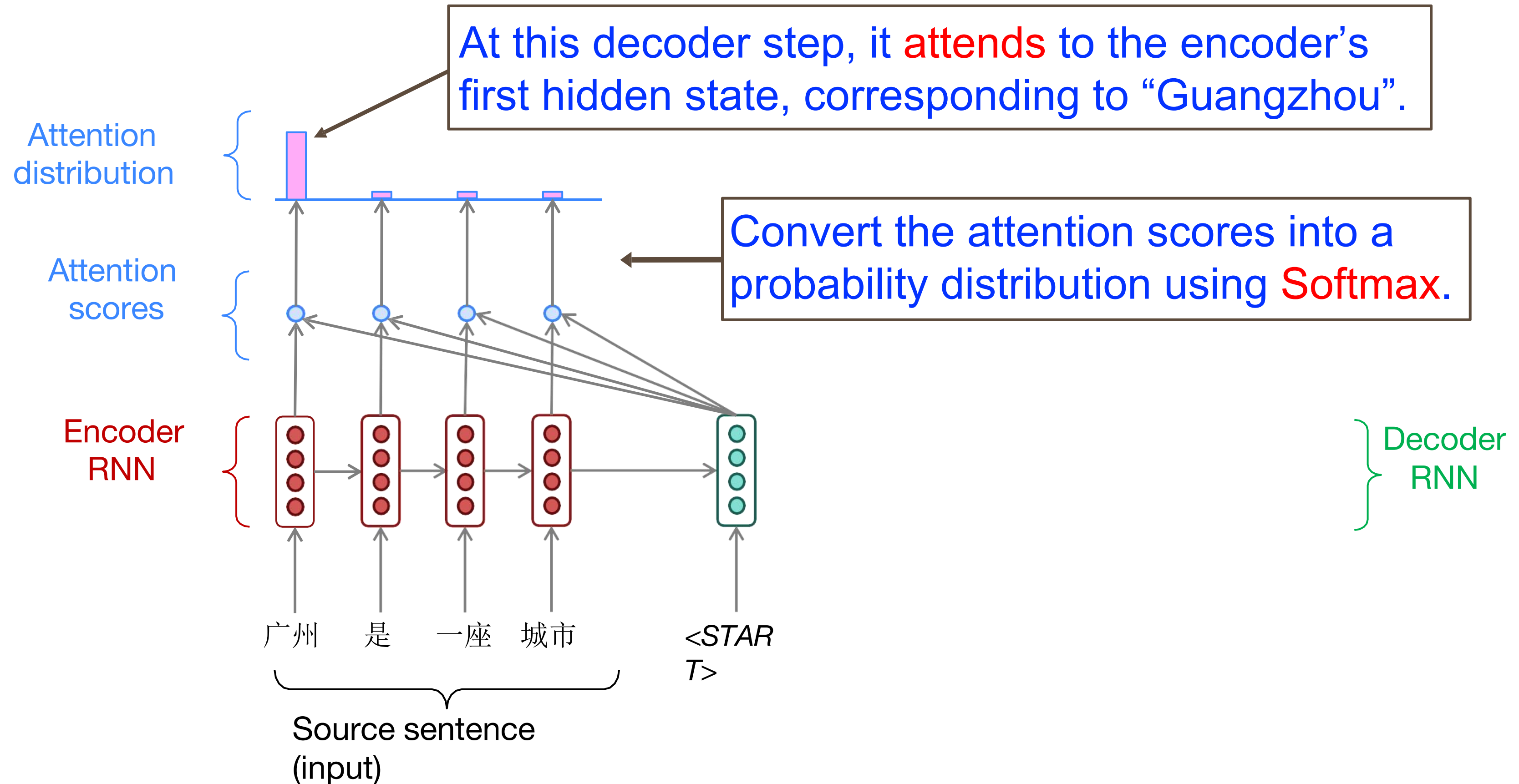
Seq2seq: Attention



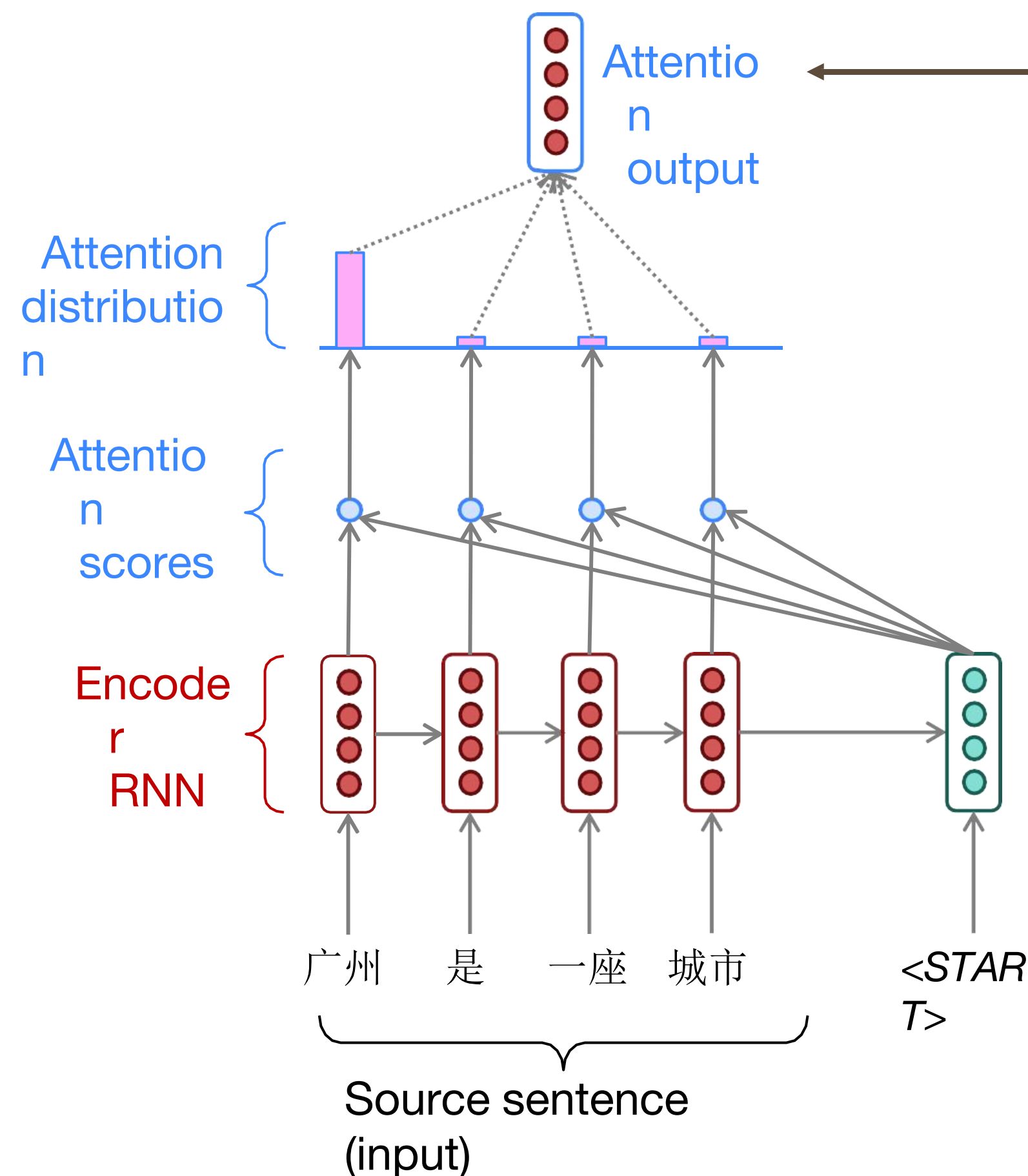
Seq2seq: Attention



Seq2seq: Attention



Seq2seq: Attention

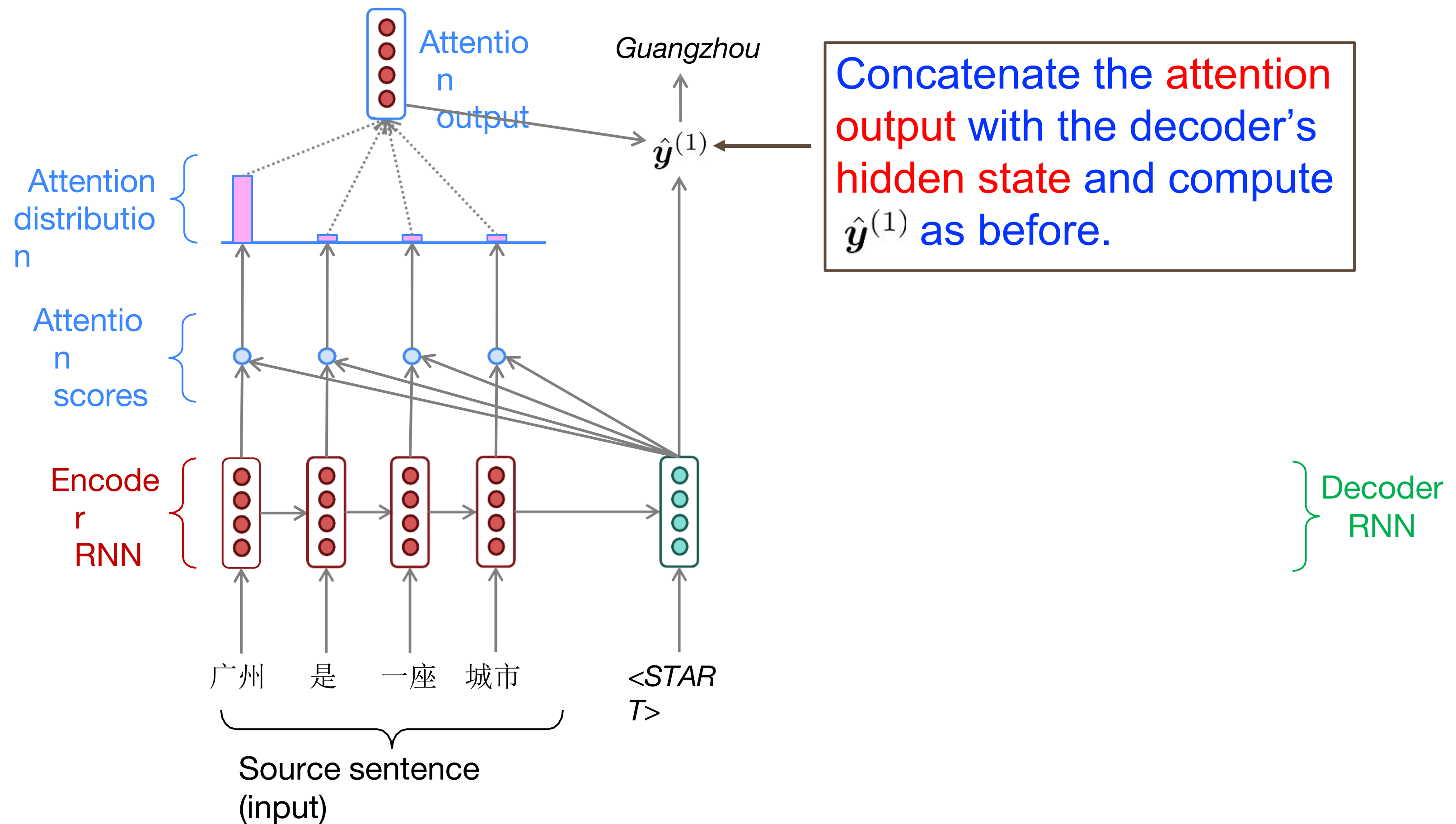


Compute a **weighted sum** of the encoder's hidden states using the **attention distribution**.

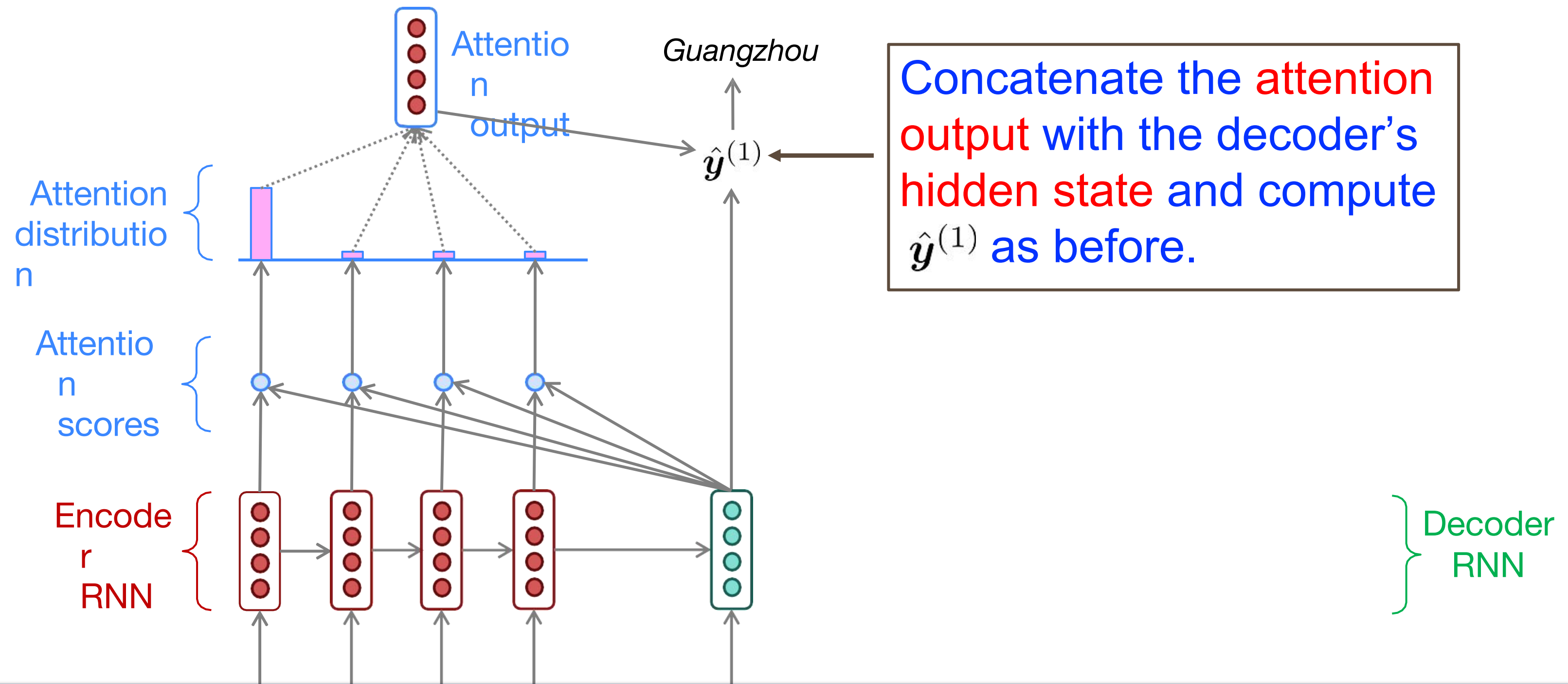
The attention output mainly contains information carried by the **highly attended hidden states**.

Decoder RNN

Seq2seq: Attention

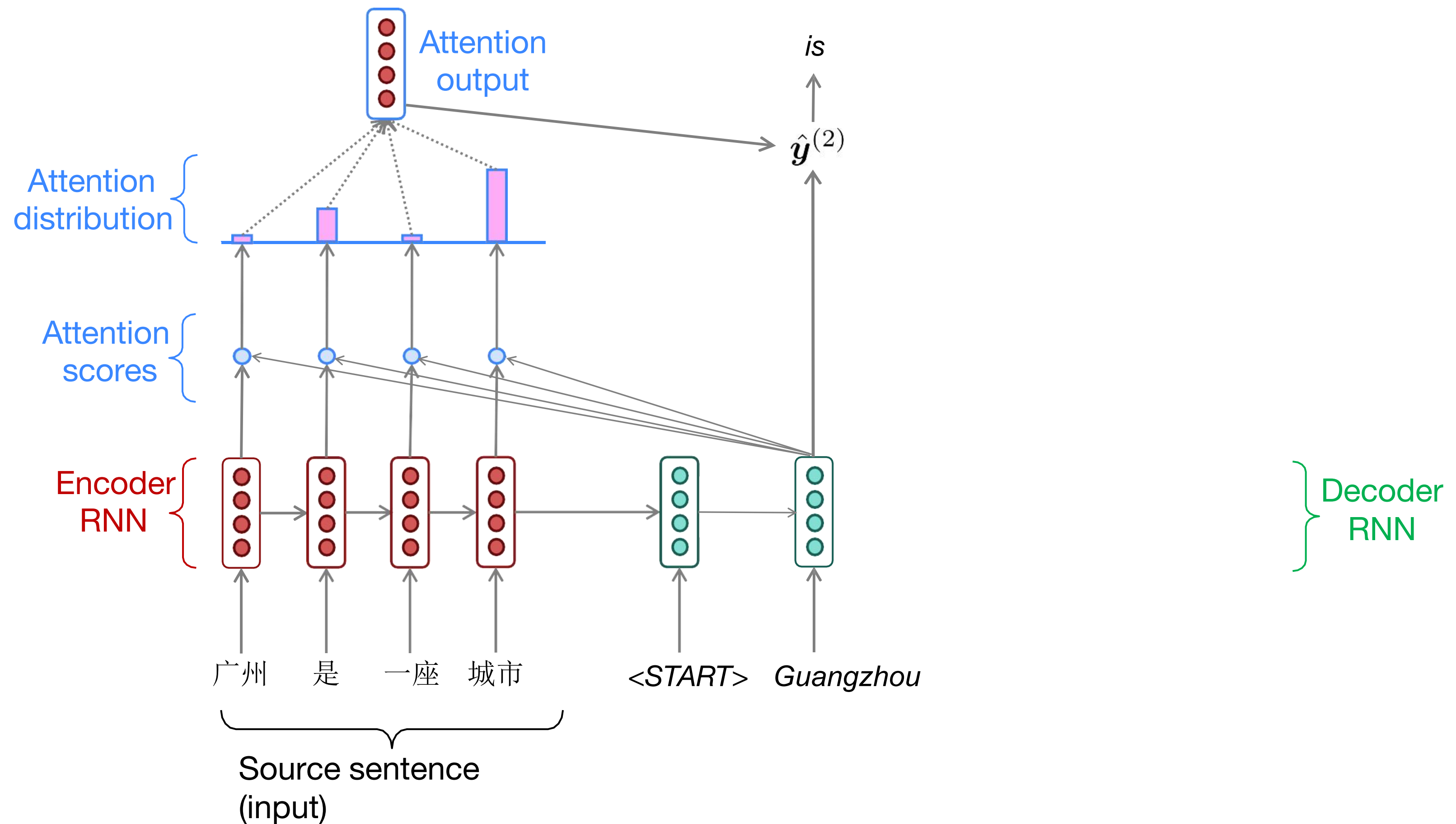


Seq2seq: Attention

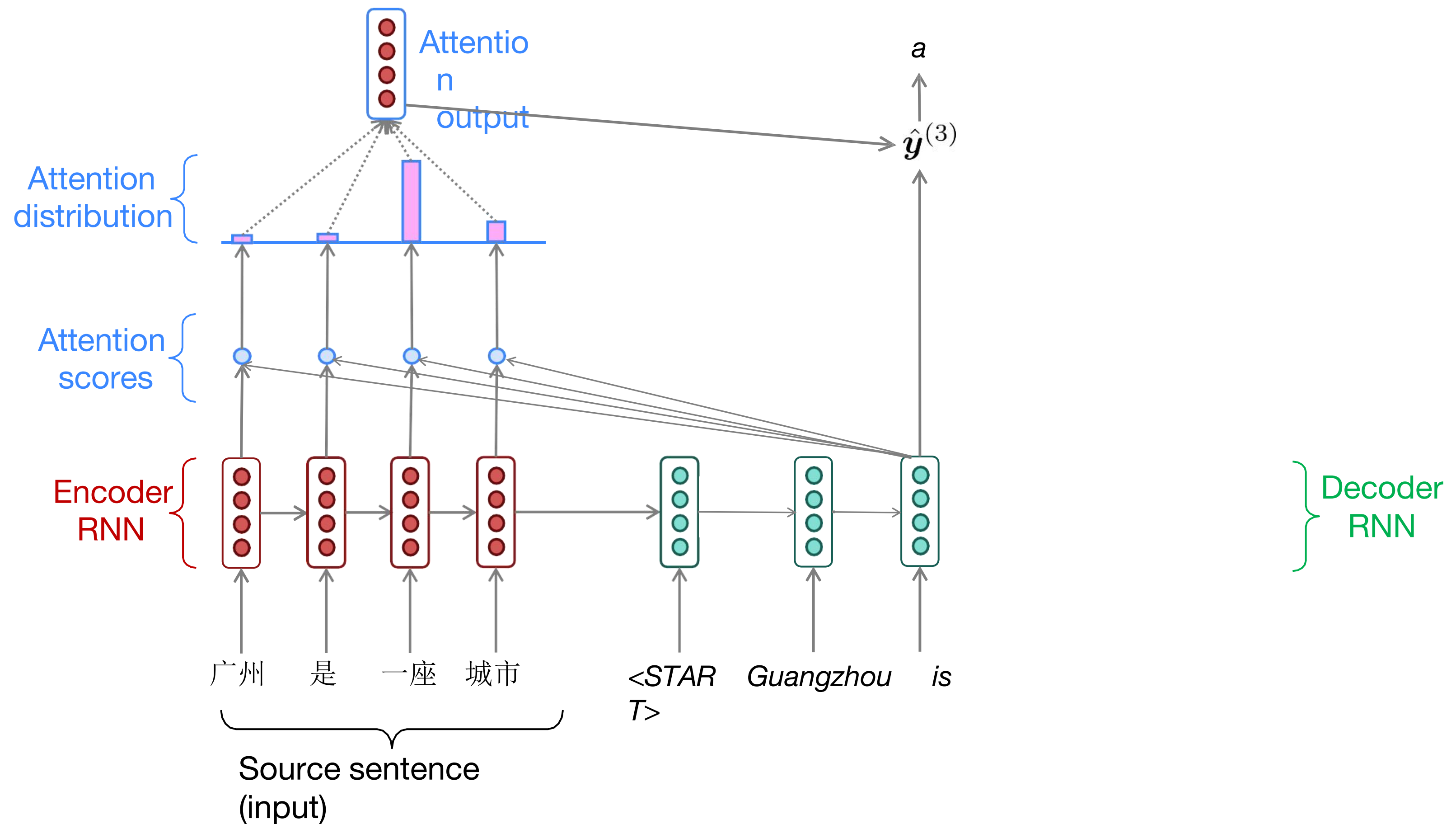


Vector concatenation is the process of joining two or more vectors end-to-end to create a single, larger vector. For example, concatenating vector $A = [1, 2]$ and vector $B = [3, 4]$ results in $[1, 2, 3, 4]$.

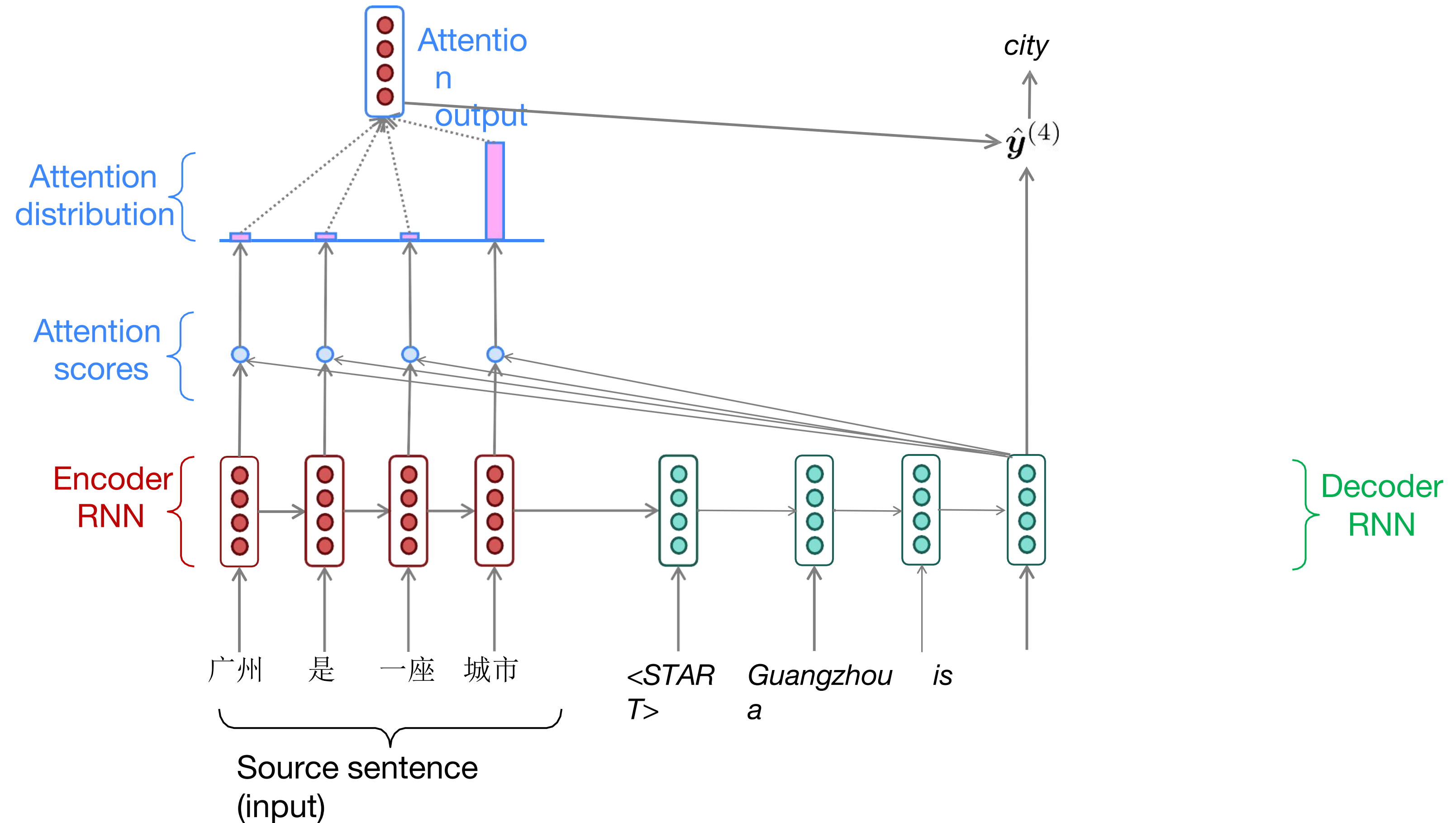
Seq2seq: Attention



Seq2seq: Attention



Seq2seq: Attention



Mathematical Description of Attention

- Encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$.
- At time step t , the decoder hidden state $s_t \in \mathbb{R}^h$.
- Get the attention score at the current time step e^t .

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- Using the **softmax** function, we get the attention distribution at that time step (the probability distribution, with a sum of **1**).

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

Mathematical Description of Attention

- Use the attention distribution α^t to compute a weighted sum of the encoder hidden states, yielding the attention output:

$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^h$$

- Concatenate the attention output with the decoder hidden state:

$$[\mathbf{a}_t; \mathbf{s}_t] \in \mathbb{R}^{2h}$$

- The subsequent process is similar to the non-attentional sequence-to-sequence (seq2seq) model.

Outline

- Language Modeling
- Recurrent Neural Networks
- RNN Language Modeling And Training
- RNN Applications And Seq2seq

RNN Limitations And The Bridge To Attention

PRACTICE & DISCUSSION

Takeaways

Outline

- Language Modeling
- Recurrent Neural Networks
- RNN Language Modeling And Training
- RNN Applications And Seq2seq

RNN Limitations And The Bridge To Attention

Practice & Discussion

TAKEAWAYS

Key Ideas

Language models predict probability distributions over next tokens.

1. The chain rule decomposes sequence probability into local predictions.

2. RNNs update a hidden state with shared parameters.

Long-distance dependencies and sequential computation limit basic RNNs.

3. *Attention* in RNNs allows the model to dynamically focus on the most relevant hidden states of the input sequence when generating each output.

Next lecture: *Attention is all you need.*

 **Learn** something **useful.**
 ***Enjoy*** **the process.**